

Dossier E5-ter — Sparsità dReLU a Scala + Località Esperti MoE

Estrazione dalla letteratura per fondazione 5M con MLP gated-dReLU ternario + MoE
E32×h128 top-8

Brief n.3 — da incollare al Ricercatore

Phase 62 — Design Gate (Cat-A)

Sparsità intrinseca 92% hidden / 79% gate (misurata)

Predictor FALSIFICATI a op-point pari (misurato)

Block-structured predictor-free BS8: 18% → 50%, BPB zero

Persistenza temporale nulla (misurata due volte)

MoE E32×h128 top-8: working-set $\approx$ i.i.d. (misurato)

Data: 2026-07-04

Sintesi fattuale dei findings

Questo dossier estrae dalla letteratura recente (preprint ultimi 18 mesi + paper peer-reviewed fondativi) i dati pubblicati su: (1) predictor di sparsità a scala, (2) dReLU in letteratura, (3) località/caching di esperti MoE. Tutti i numeri sono etichettati **[MISURATO]** o **[PROIETTATO]** e associati alla rispettiva baseline. Non contiene raccomandazioni — la decisione di design è a valle, con i dati Cat-A di Phase 62.

Punti chiave (solo fatti, no raccomandazioni)

- **Predictor di sparsità: 6/7 paper li USANO, 1/7 è esplicitamente predictor-free.** DejaVu (ICML 2024), PowerInfer (SOSP 2024), PowerInfer-2 (2024), Turbo Sparse (2024), ProSparse (COLING 2025), SparseInfer (DAC 2025) usano tutti un predictor (5 training-based tipo DejaVu 2-layer MLP, 1 training-free tipo SparseInfer sign-bit XOR). **ReLU Strikes Back (Apple, ICLR 2024)** è l'unico esplicitamente predictor-free: *"there is no need to train a separate prediction module that complicates the computation graph, and using efficient ReLU layers can be enough"*. Il nostro finding "predictor falsificati a op-point pari" è allineato a ReLU Strikes Back e si spinge oltre (block-structured BS8 predictor-free).
- **dReLU: ALLINEATO FORTE con Turbo Sparse.** Turbo Sparse (Song et al., arXiv 2406.05955, giu 2024) usa dReLU con formula $\max(0, x \cdot W_{\text{gate}}) * \max(0, x \cdot W_{\text{up}})$ — ReLU su entrambi i lati del gated MLP (vs SwiGLU = SiLU(gate) * up). dReLU induce ~90% FFN sparsity su Mistral-7B (vs 70% di ReGLU) e 97% su MoE Mixtral-47B. Ricetta: ReLUfication post-hoc con continued pretraining (150B tokens, <1% dei 15T tipici di pretraining) + small from-scratch test a 300M. Il nostro setup (dReLU + 92% hidden / 79% gate sparse) è coerente con Turbo Sparse (90% FFN).
- **Località MoE: ALLINEATO FORTE con Local Routing Consistency.** Liang et al. 2025 (arXiv 2505.16056) misurano direttamente località di routing su 20 MoE LLM. **Mixtral-8x7B (E=8, k=2, base-rate i.i.d. 50%): SRP@16=49.27% $\approx$ i.i.d. base-rate** (gap -0.7 pp, essenzialmente i.i.d.). Long-range (m=256, cache 2×k): SCH=61.85% (+12 pp sopra i.i.d. — località marginale). Setup più vicino al nostro E32/k8: PowerMoE (E=40, k=8) SRP@16=54.69% vs i.i.d. 40% (+14.7 pp). **Modelli granular con shared experts hanno località PIÙ BASSA:** DeepSeekMoE (E=64, k=6, +2 shared) SRP@16=36.79% vs i.i.d. 18.75% (+18 pp ma più vicino a i.i.d. in termini relativi). Il nostro setup E32 granular è affine a PowerMoE → aspettiamo località marginale.
- **Spiegazione architetturale del finding 'working-set $\approx$ i.i.d.': Switch Transformer (Fedus et al., JMLR 2022) adotta auxiliary load balancing loss che forza uniform routing → working-set $\approx$ i.i.d. BY DESIGN.** La maggior parte dei MoE moderni eredita questa pratica → il nostro finding è conseguenza architetturale, non anomalia. **TUTTI i 7 paper MoE che raggiungono speedup misurabili** (Mixtral-offloading, Pre-gated MoE, MoE-Infinity, EdgeMoE, SiDA-MoE, Fiddler, Local Routing Consistency) usano **predizione** (next-layer,

cross-layer statistical, hash+LSTM, trace-based) — NESSUNO si affida a puro working-set locality / LRU. Questo VALIDA direttamente il nostro finding "località solo da predizione".

- **Zona bianca confermata.** Nessun paper testa: (a) combinazione 5M + dReLU + ternary weights (Turbo Sparse è fp16/INT4, BitNet b1.58 è ternary+ReLU non dReLU); (b) block-structured predictor-free BS8 esplicito (concept più vicino: PowerInfer-2 neuron cluster adattivo, ReLU Strikes Back row-skipping, SparseInfer sign-bit packing 32-elementi); (c) persistenza temporale = mask overlap token-to-token (ReLU Strikes Back misura aggregated sparsity, metrica diversa); (d) granular MoE E32/h128 su consumer x86 single-thread. **MoE-Infinity** (ICML 2024) è l'unico paper che claim skewed-reuse intra-request → [CONTRADDIZIONE PARZIALE] col nostro finding, ma lo stesso paper ammette "after processing multiple requests, the skew disappears" → [ALLINEATO cross-request].
- **Nessuna contraddizione formale con i nostri numeri.** Speedup pshufb-LUT 4.2-5× vs fp32 (dossier E5-bis) è allineato a bitnet.cpp/T-MAC/DeepGEMM. dReLU + 92% sparse è allineato a Turbo Sparse 90%. Working-set $\approx$ i.i.d. è allineato a Local Routing Consistency. **Unica contraddizione sostanziale:** MoE-Infinity su intra-request skewed-reuse (ma cross-request allineato). **Unica tensione concettuale:** ReLU Strikes Back misura aggregated sparsity > random su OPT-6.7B → metrica diversa dal nostro "persistenza temporale nulla" (mask overlap token-to-token); aggregated può essere > random anche con overlap=0.

1. Domanda 1 — Predictor di sparsità a scala

Sette paper identificati. Coprono: DejaVu (ICML 2024, predictor 2-layer MLP), PowerInfer (SOSP 2024, adaptive predictor), PowerInfer-2 (2024, polymorphic neuron engine), Turbo Sparse (giu 2024, dReLU + predictor), ProSparse (COLING 2025, ReLUfication + L1 progressiva), ReLU Strikes Back (Apple, ICLR 2024, predictor-free esplicito), SparseInfer (DAC 2025, training-free sign-bit XOR). **6/7 USANO predictor; 1/7 (ReLU Strikes Back) è esplicitamente predictor-free.** Scala minima testata: 300M (Turbo Sparse micro-exp). Nessun paper a 5M.

1.1 DejaVu — Contextual Sparsity for Efficient LLMs

Citazione: [1] Liu, Z.; Wang, J.; Dao, T.; Zhou, T.; Yuan, B.; Song, Z.; Shrivastava, A.; Zhang, C.; Tian, Y.; Ré, C.; Chen, B. (Stanford; UW; Together AI), *Deja Vu: Contextual Sparsity for Efficient LLMs at Inference Time*, ICML 2024; arXiv:2310.17157. URL: <https://arxiv.org/abs/2310.17157> — codice: <https://github.com/FMInference/DejaVu>.

Metodo. Predictor SÌ: per ogni MLP block e ogni Attention block, viene addestrato un **piccolo MLP a 2 strati fully-connected** che predice la sparsità contestuale. Training usa **500 campioni casuali dal C4 training set**. Reason: HNSW richiede >10ms (vs 0.2ms MLP computation su OPT-175B), quindi scelgono MLP per sfruttare GEMM veloce su GPU. Attached a ciascun layer (predictor per MLP block + predictor per Attention block, separati). Sparsità intrinseca sfruttata parzialmente (OPT è ReLU-native: 95% MLP parameters zeroable, 80% attention heads silenziale), ma la **selezione dinamica richiede il predictor**. Activation function: usa modelli OPT (ReLU nativo). ReLUfication: N/A (già ReLU nativo).

Scala e architettura. Parametri: 1B+ (OPT-1.3B, OPT-13B, OPT-30B, OPT-66B, OPT-175B; BLOOM; LLaMA citati). Architettura: Transformer decoder-only (OPT, BLOOM). Layer target: MLP block (85% sparsity) + Attention heads (50% sparsity), separatamente o congiuntamente.

Accuratezza predictor [MISURATO]. Validation accuracy MLP: >99% nei layer shallow, ~93% nei layer finali. Validation accuracy Attention: ~93% nei layer medi, ~99% nei layer shallow e deep. Recall/TPR/FPR: non riportati in forma numerica (solo "validation accuracy"). Definizione usata: accuratezza di classificazione binaria dei neuroni attivi. Baseline: oracle (densa). DejaVu non ha accuracy loss a 85% MLP + 50% attention sparsity.

Costo predictor. Parametri aggiuntivi: non riportati come numero assoluto o %. Latency overhead: NON riportato come numero. Mitigato via **asynchronous lookahead predictor** (Section 4.3): il predictor per il layer k+1 viene eseguito in parallelo al calcolo del layer k. *"Sparse prediction overhead may easily increase the end-to-end latency rather than reduce it despite the reduction in FLOPs"* — explicit concern. Memory overhead: non riportato. Cita HNSW >10ms vs 0.2ms MLP per giustificare la scelta del MLP predictor.

Speedup netto [MISURATO], baseline = FasterTransformer (FT, NVIDIA C++/CUDA) + Hugging Face (HF), FP16, batch=1, single-user decoding. Speedup: 1.8–2× vs FasterTransformer, 4.8–6× vs Hugging Face, su 8×A100 80GB, OPT-175B, prompt length 128/256/512/1024. Speedup algoritmico puro: ~7× parameter reduction a 85% sparsity (teorico), ma wall-clock è 1.8–2× a causa di overhead e memory-bound. Punto di sweet spot: ~75% sparsity.

Struttura a blocchi / coerenza temporale. Block-structured (BS8/BS16): NON documentato. *"Block size"* menzionato è solo quello della **memoria GPU** (128 bytes), non della sparsità. Coerenza temporale maschere: NON misurata. Esiste il concetto di *"slowly evolving embedding"* (similitudine coseno tra layer consecutivi ≥ 0.95), ma è **cross-layer (spaziale)**, NON **cross-token (temporale)**. Una sola menzione: *"one particular challenge of sparse prediction in attention blocks is attention's dependence on previous tokens"* — ma non misura overlap maschere tra token.

Regime. Hardware: 8×A100 80GB con NVLink (tensor parallelism). Batch size: 1 (token generation phase, single-user). Studio esplicito anche su batch più grandi (Section 5.2 — *Contextual Sparsity for Larger Batches*). Memory-bound vs compute-bound: MEMORY-BOUND esplicito. Tabella 1: *"Token Generation 128 [...] I/O Latency 2600 ms"* vs *"Prompting 128 [...] I/O Latency 20.6 ms"* — il decoding è 130× più lento per I/O. *"Token generation phase easily dominates the end-to-end inference time"*.

Cosa copre del nostro caso. Memory-bound framing (forte); sparsità intrinseca ReLU-style ~85-95% (coerente col nostro 92%); predictor accuracy >93% su larga scala; speedup 1.8-2× su hardware memory-bound; architettura Transformer decoder. **Cosa NON copre.** Scala 5M (DejaVu è 1.3B-175B); dReLU (DejaVu usa ReLU nativo); pesi ternari; **block-structured predictor-free BS8; persistenza temporale nulla; confronto predictor vs no-predictor a op-point parity**; CPU consumer.

Contraddizioni con le nostre misure. Customer finding *"predictor falsificati"*: DejaVu USA un predictor (2-layer MLP) e ne riporta validation accuracy 93-99% → NON testa esplicitamente *predictor vs no-predictor a op-point parity*. → [N/A] — non contraddice direttamente il nostro

finding, ma l'intera architettura DejaVu è costruita attorno al predictor. Customer finding "*block-structured predictor-free BS8 18% → 50%, BPB zero*": DejaVu non considera questa opzione. [N/A] — territorio non coperto. Customer finding "*persistenza temporale nulla*": DejaVu non misura la persistenza temporale. [N/A]. **[POTENZIALE CONTRADDIZIONE]**: DejaVu dimostra 2× wall-clock speedup con predictor su OPT-175B. Customer non vede benefit del predictor a 5M scale. Possibile spiegazione (non in DejaVu): a 175B con 8×A100, il predictor ammortizza I/O cost di 175B pesi; a 5M su CPU, l'I/O cost è marginale → il predictor aggiunge overhead senza risparmiare abbastanza. **Ipotesi non verificata nel paper.**

1.2 PowerInfer — Fast LLM Serving with Consumer-grade GPU

Citazione: [2] Song, Y.; Mi, Z.; Xie, H.; Chen, H. (SJTU-IPADS), *PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU*, SOSP 2024; arXiv:2312.12456. URL: <https://arxiv.org/abs/2312.12456> — codice: <https://github.com/SJTU-IPADS/PowerInfer>.

Metodo. Predictor SÌ: "*Adaptive predictors*" — MLP predictor ispirato a DejaVu (2-layer), ma con dimensionamento **adattivo** dello hidden layer in base alla skewness interna di ogni layer: layer molto skew → hidden più piccolo (fino a cadere sotto 95% accuracy); layer poco skew → hidden più grande. Architettura predictor: MLP a 3 strati (input, hidden adattivo, output); dimensione hidden iteratively tuned. Sparsità intrinseca sfruttata parzialmente: hot neurons preloaded su GPU offline (no predictor necessario per loro); cold neurons richiedono il predictor online. Activation function: usa modelli esistenti (OPT=ReLU, LLaMA=ReLU/SiLU). ReLUfication: N/A.

Scala e architettura. Parametri: 7B-175B (OPT 7B-175B, LLaMA 7B-70B, Falcon-40B). Architettura: Transformer decoder-only. Layer target: FFN/MLP neurons (singolarmente); attention heads non sono il focus primario.

Accuratezza predictor [MISURATO]. Accuracy: >93% citato (referenziato da DejaVu). Adaptive sizing mantiene **≥95% accuracy** come soglia. Recall/TPR/FPR: non riportati in forma numerica. Baseline: oracle (dense).

Costo predictor [MISURATO]. Parametri aggiuntivi: **~10% dei parametri totali del LLM** ("*PowerInfer effectively limits predictor parameters to a mere 10% of the total LLM parameters*"). Latency overhead: **<10% del tempo di inferenza** (Figure 17, end-to-end predictor overhead). "*On average, the execution of predictors constitutes less than 10% of the total inference time in PowerInfer*". Memory overhead: predictors pre-loaded su GPU. Figura 9: passaggio da 480MB a 320MB del predictor fa cadere accuracy da 92% a 84% → winogrande accuracy da 72.77% a 67.96%.

Speedup netto [MISURATO], baseline = llama.cpp (non-quantizzato) + llama.cpp (quantizzato). Speedup: fino a **11.69× vs llama.cpp (non-quantizzato)**, 8.00× vs llama.cpp (quantizzato); 7.23× average su LLaMA-70B; 13.20 tok/s avg, 29.08 tok/s peak su RTX 4090 con OPT-175B. vs server-grade A100: solo 18% più lento di A100 (che può contenere tutto il modello). Speedup algoritmico puro: non separato esplicitamente; citato che >99.5% del tempo di decoding è I/O transfer da CPU a GPU → speedup ammortizza I/O.

Struttura a blocchi / coerenza temporale. Block-structured (BS8/BS16): NON documentato. PowerInfer ragiona a livello di singolo neurone (row/column di weight matrix), non di blocchi BS8. Coerenza temporale maschere: NON misurata. Implicita solo la *"power-law distribution across inputs"* (hot neurons consistent across inputs) — ma è distribuzione su ensemble di input, NON maschere consecutive.

Regime. Hardware: PC consumer-grade con singola GPU (RTX 4090 24GB, RTX 2080Ti 11GB); CPU host (Intel/AMD non specificati). Batch size: 1 (single-user, local deployments esplicitati). *"Local deployments focus on low latency in processing small batches"*. Memory-bound vs compute-bound: MEMORY-BOUND esplicito. *"Over 99.5% of processing time is consumed by transferring LLM weights from CPU to GPU, significantly impacting overall latency"* (Figura 4b).

Cosa copre del nostro caso. Memory-bound CPU-GPU offload framing (fortissimo); single-user batch=1; predictor overhead <10% misurato; consumer-grade hardware; sparsità intrinseca ReLU ~90%. **Cosa NON copre.** Scala 5M (PowerInfer è 7B-175B); dReLU (usa modelli esistenti); pesi ternari; **block-structured predictor-free BS8; persistenza temporale; confronto predictor vs no-predictor a op-point parity**; CPU-only single-thread (PowerInfer è CPU+GPU hybrid).

Contraddizioni con le nostre misure. Customer *"predictor falsificati"*: PowerInfer USA un predictor adaptive-MLP con overhead <10% e accuracy $\geq 95\%$ → NON testa predictor vs no-predictor a parity. [N/A] — ma l'overhead <10% è un numero interessante per quantificare il costo del predictor in customer's regime (benché su GPU non CPU). Customer *"block-structured predictor-free"*: PowerInfer ragiona per singolo neurone, non per BS8. [N/A]. Customer *"persistenza temporale nulla"*: non misura. [N/A]. **[POTENZIALE CONTRADDIZIONE]**: PowerInfer ottiene $11.69\times$ speedup con predictor su 7B-175B. Customer non vede benefit a 5M. Possibile spiegazione (non in PowerInfer): il speedup di PowerInfer dipende dal memory-bandwidth saving su 7B+ pesi da trasferire CPU → GPU; a 5M su CPU, il peso del transfer è nullo → predictor overhead dominante.

1.3 PowerInfer-2 — Fast LLM Inference on a Smartphone

Citazione: [3] Xue, Z.; Song, Y.; Mi, Z.; Chen, L.; Xia, Y.; Chen, H. (SJTU-IPADS), *PowerInfer-2: Fast Large Language Model Inference on a Smartphone*, arXiv:2406.06282. URL: <https://arxiv.org/abs/2406.06282> — sito: <https://powerinfer.ai/v2>.

Metodo. Predictor SÌ (riusa PowerInfer predictor). Durante decoding, *"invokes a predictor to identify which neurons will be activated before initiating computations"*. Architettura predictor: MLP predictor come PowerInfer (adattivo), una per FFN block. Sparsità intrinseca: hot neurons preloaded in cache; cold neurons richiedono predictor. Innovazione chiave: **polymorphic neuron engine** — prefill usa *"big neuron cluster"* (per NPU su large matrix), decoding usa *"small neuron cluster"* (per CPU cores su batch=1). Activation function: usa modelli Llama-2 (SiLU/SwiGLU) e TurboSparse-Mistral/Mixtral (dReLU). ReLUfication: N/A (usa modelli esistenti, tra cui TurboSparse).

Scala e architettura. Parametri: 7B-47B (Llama-2 7B/13B, TurboSparse-Mistral-7B, TurboSparse-Mixtral-47B). Architettura: Transformer decoder-only dense + MoE (Mixtral-47B = 8-expert MoE). Layer target: FFN/MLP (incl. expert FFN nel MoE).

Accuratezza predictor. Non riportato come numero separato in PowerInfer-2 (rimanda a PowerInfer). Recall/TPR/FPR: non riportati. **[MISURATO indirettamente]** (eredita da PowerInfer).

Costo predictor. Parametri aggiuntivi: non riportati separatamente. Latency overhead: mitigato via *"segmented neuron caching"* + *"fine-grained neuron-cluster-level pipelining"* che *"conceal the overhead caused by I/O operations"*. Memory overhead: ~40% memory reduction mantenendo speed comparabile a llama.cpp/MLC-LLM per modelli che entrano in memoria. **[MISURATO]** indirettamente.

Speedup netto [MISURATO], baseline = llama.cpp + LLM in a Flash. Speedup: fino a **29.2× vs llama.cpp** (decoding, TurboSparse-Mixtral-47B); 25.4× average; 3.94× average vs LLM in a Flash; 21.2× vs llama.cpp per TurboSparse-Mixtral-47B (11.68 tok/s su smartphone). Hardware: OnePlus 12 / OnePlus Ace 2.

Struttura a blocchi / coerenza temporale. Block-structured (BS8/BS16): PARZIALMENTE. *"Neuron cluster"* è un'unità a granularità fine, la cui dimensione è determinata dalle caratteristiche XPU/memoria/I/O — NON è un BS8 fisso ma un cluster adattivo. *"A neuron cluster consists of multiple neurons, whose number is determined by the characteristics of the XPU's, memory, and I/O"*. Coerenza temporale maschere: NON misurata direttamente. *"Segmented neuron caching"* e *"in-memory cache that maintains the most frequently used neurons"* implica caching temporale, ma non misura overlap maschere consecutive. Numeri: nessun numero su overlap temporale.

Regime. Hardware: smartphone OnePlus 12 (Snapdragon 8 Gen 3, 24GB DRAM, UFS 4.0) + OnePlus Ace 2 (16GB). XPU eterogeneo: big.LITTLE CPU + GPU + NPU Qualcomm. Batch size: 1 (decoding single-user su smartphone). Explicit: *"decoding stage, which has a batch size of one"*. Memory-bound vs compute-bound: MEMORY + I/O-BOUND esplicito. *"I/O activities a frequent bottleneck in LLM inference on mobile devices"* + *"single command queue"* limitation.

Cosa copre del nostro caso. Memory+I/O bound su edge device (molto simile al nostro regime CPU consumer); single-user batch=1; neuron-cluster come unità di sparsità (prossimo al BS8 concettualmente, anche se adattivo non fisso); MoE layer target (Mixtral-47B ha 8 expert FFN, vicino al nostro *località esperti MoE*). **Cosa NON copre.** Scala 5M (P-2 è 7B-47B); dReLU ternary (usa fp16/INT4); **predictor vs no-predictor a op-point parity**; CPU consumer x86 single-thread; **persistenza temporale numerica.**

Contraddizioni con le nostre misure. Customer *"predictor falsificati"*: P-2 usa predictor → **[N/A]** su falsificazione. Customer *"block-structured predictor-free BS8 18% → 50%, BPB zero"*: P-2 ha *"neuron cluster"* (concetto affine) ma **NON è predictor-free** (usa predictor per selezionare cluster) e **NON misura BPB-zero cost.** **[N/A]**. Customer *"persistenza temporale nulla"*: P-2 usa segmented caching (implica caching temporale) ma **NON misura overlap temporale.** **[N/A]**. **[POTENZIALE CONTRADDIZIONE]**: P-2 ottiene 29.2× su smartphone con predictor; customer non vede benefit a 5M CPU. Possibile spiegazione: P-2 ammortizza I/O storage → DRAM su 47B model; a 5M su DRAM il transfer è nullo.

1.4 Turbo Sparse — Achieving LLM SOTA with Minimal Activated Parameters

Citazione: [4] Song, Y.; Xie, H.; Zhang, Z.; Wen, B.; Ma, L.; Mi, Z.; Chen, H. (SJTU-IPADS; Tsinghua), *Turbo Sparse: Achieving LLM SOTA Performance with Minimal Activated Parameters*, arXiv:2406.05955, giugno 2024. URL: <https://arxiv.org/abs/2406.05955> — modelli: <https://huggingface.co/PowerInfer>.

Metodo. Predictor SÌ in inference. *"During the SFT phase, we incorporate a predictor module for each FFN block. Notably, for the TurboSparse-Mixtral-47B, we train predictors for each expert."* Architettura predictor: per-FFN module (dettagli architettura non forniti, integrato in PowerInfer per inference). Per-MoE-expert predictor su Mixtral. Sparsità intrinseca SENZA rete ausiliaria: SÌ. dReLU induces sparsità intrinseca ~90% senza predictor. Il predictor è solo per **selezione dinamica** di quale 10% calcolare.

dReLU formula (CHIAVE per la Domanda 2). $\text{Combined_dReLU}(x) := \max(0, x \cdot W_{\text{gate}}) * \max(0, x \cdot W_{\text{up}})$ — ReLU applicato a **ENTRAMBI** gate E up projection (vs SwiGLU = SiLU(gate) * up; vs ReGLU = ReLU(gate) * up). Doppia ReLU → 90% FFN sparsity (vs 70% di ReGLU). ReLUfication: SÌ — continued pretraining con dReLU (150B tokens, <1% dei 15T tipici di pretraining). From-scratch test: piccolo esperimento a 300M params, 5B tokens, su fineweb → dReLU match SwiGLU performance.

Scala e architettura. Parametri: 300M (small experiment) + 7B (TurboSparse-Mistral-7B) + 47B (TurboSparse-Mixtral-47B, MoE 8-expert). Architettura: Transformer decoder-only dense (Mistral) + MoE (Mixtral 8 expert). Layer target: FFN/MLP (incl. expert FFN).

Accuratezza predictor. TPR/FPR/Recall: non riportati numericamente. **[MISURATO indirettamente]** (predictor integrato in PowerInfer, vedi 1.2). Baseline: oracle (dense).

Costo predictor. Parametri aggiuntivi: non riportati separatamente. Latency overhead: non riportato (eredita da PowerInfer <10%). Memory overhead: non riportato. **[NON RIPORTATO]**.

Speedup netto [MISURATO], baseline = llama.cpp (con modelli originali non sparsificati). Speedup: 2-5× **decoding speedup** overall; CPU-only 2.08-2.81× vs llama.cpp (PC-2080Ti, PC-Laptop); hybrid GPU-CPU 2.32-4.64× vs llama.cpp; mobile 11 tok/s su TurboSparse-Mixtral-47B via PowerInfer-2. Hardware training: 64 A800-80G GPUs.

Struttura a blocchi / coerenza temporale. Block-structured (BS8/BS16): NON documentato. Coerenza temporale maschere: NON misurata. Numeri: nessun numero su overlap temporale o block structure. **MoE-specific finding:** *"In MoE models, we further improve the sparsity in TurboSparse-Mixtral-47B, originally introduced due to expert routing, from 75% to 97%"* → MoE FFN experts restano sparsamente attivi anche dopo routing.

Regime. Hardware inference: PC-Laptop (Intel i9-14900HX, 32GB DRAM, RTX 4090 16GB, PCIe 4.0), PC-2080Ti (i7-12700K, 64GB, RTX 2080Ti 11GB, PCIe 3.0), OnePlus-12 (Snapdragon 8 Gen 3, 24GB DRAM, UFS 4.0). Batch size: 1 (decoding single-user, esplicitato in sezione mobile). Memory-bound vs compute-bound: implicito memory-bound (eredita framing PowerInfer/PowerInfer-2); CPU-only è ampiamente memory-bound.

Cosa copre del nostro caso. COPRE FORTE: dReLU activation function (DIRECT MATCH con customer "*MLP gated-dReLU ternary*"); sparsità intrinseca 90% (DIRECT MATCH con customer 92% hidden / 79% gate); MoE expert FFN sparsity (DIRECT MATCH con customer "*località esperti MoE*"); small-scale validation (300M, anche se non 5M); CPU inference 2-2.8× (vicino al regime customer). **Cosa NON copre.** Scala 5M (TurboSparse è 300M/7B/47B); **pesi ternari** (TurboSparse è fp16/INT4); **block-structured predictor-free BS8**; **persistenza temporale nulla**; **confronto predictor vs no-predictor a op-point parity**; predictor architettura dettagli non riportati.

Contraddizioni con le nostre misure. Customer "*dReLU + 92% hidden sparse / 79% gate sparse*": **[ALLINEATO FORTE]** — TurboSparse riporta 90% FFN sparsity con dReLU su Mistral-7B, 97% su MoE Mixtral-47B. Customer's 92% hidden / 79% gate è coerente con TurboSparse's 90% (dReLU → ~90% su entrambi i lati). **Convergenza diretta.** Customer "*predictor falsificati a op-point parity*": TurboSparse **USA** predictor (SFT phase, integrato con PowerInfer). **[N/A]** su falsificazione. Ma: TurboSparse NON confronta predictor vs no-predictor a op-point parity — il predictor è dato come necessario per inference. **[POTENZIALE CONTRADDIZIONE]:** TurboSparse claim 2-5× speedup con predictor; customer non vede benefit. Possibile spiegazione: TurboSparse's 2-5× è vs original-dense Mistral (senza alcuna sparsità), non vs *dReLU+block-skip senza predictor* — quindi la sparsità intrinseca (dReLU) da sola potrebbe dare gran parte del speedup, predictor aggiunge poco se la selezione può avvenire block-level. Customer "*block-structured predictor-free BS8*": TurboSparse non considera questa opzione. **[N/A]**. Customer "*persistenza temporale nulla*": non misura. **[N/A]**.

1.5 ProSparse — Introducing and Enhancing Intrinsic Activation Sparsity

Citazione: [5] Song, C.; Han, X.; Zhang, Z.; Hu, S.; Shi, X.; Li, K.; Chen, C.; Liu, Z.; Li, G.; Yang, T.; Sun, M. (Tsinghua), *ProSparse: Introducing and Enhancing Intrinsic Activation Sparsity within Large Language Models*, COLING 2025; arXiv:2402.13516. URL: <https://arxiv.org/abs/2402.13516> — modelli: <https://huggingface.co/SparseLLM/prosparse-llama-2-7b> e [prosparse-llama-2-13b](https://huggingface.co/SparseLLM/prosparse-llama-2-13b).

Metodo. Predictor SÌ per la via "*approximate*", NO per la via "*accurate*". Due algoritmi testati: **Approximate** = PowerInfer (DejaVu-style 2-layer MLP predictor, training-based). **Accurate** = 2 GPU sparse operators (input-side sparsity + output-side sparsity), **NO predictor** — calcolo denso + skip degli zeri noti dall'output di ReLU. Architettura predictor: DejaVu-style (ereditato da PowerInfer). Sparsità intrinseca SENZA rete ausiliaria: SÌ nella via "*accurate*" (no predictor). NO nella via "*approximate*" (richiede predictor). Activation function: ReLU (sostituita da SwiGLU/SiLU) + FATReLU threshold shifting ($T > 0$). ReLUfication: SÌ, post-hoc. 3 step: (1) activation substitution (Swish → ReLU) + continual training; (2) **progressive L1 sparsity regularization** con fattore che aumenta lungo curva sinusoidale in stadi multipli; (3) activation threshold shifting (FATReLU). Innovazione chiave: L1 reg progressiva (vs fissa) → evita shift radicale, mantiene performance.

Scala e architettura. Parametri: 7B, 13B (LLaMA2-7B, LLaMA2-13B). Architettura: Transformer decoder-only (LLaMA2, gated FFN con Swish nativo). Layer target: FFN/MLP.

Accuratezza predictor. Recall: definito come *"average ratio of correctly predicted activated elements among all truly activated elements in x1"*. Numeri in Table 2 non completamente estraibili dal HTML (tabella renderizzata parzialmente); ProSparse-7B: alta recall implicita dall'alto speedup. Predicted sparsity: definita; misurata per ReluLLaMA, Vanilla ReLU, ProSparse (senza e con threshold shifting). **[MISURATO]** (Table 2), ma numeri non tutti estraibili. Baseline: ReluLLaMA-7B (Zhang et al. 2024), Vanilla ReLU, Shifted ReLU, Fixed L1.

Costo predictor. Parametri aggiuntivi: non riportati separatamente (eredita da PowerInfer). Latency overhead: non riportato. Memory overhead: non riportato. **[NON RIPORTATO]**.

Speedup netto [MISURATO], baseline = llama.cpp (no sparsity) + dense GPU operators. Speedup approximate (PowerInfer): *"no less than 3× vs llama.cpp"* across all methods; ProSparse-LLaMA2-13B *"up to 4.65×"* (citato in SparseInfer paper che lo riferisce). Speedup accurate (sparse GPU operators): single-step speedup **2.44×** (step 2: fused ReLU + elementwise) e **1.70×** (step 3: sparse matrix-vector multiply) — best per modelli grandi. Speedup algoritmico puro: FLOPS reduction corrispondente alla sparsità ~89% → ~9× FLOPS ridotto, ma wall-clock molto minore (3-4.65×).

Struttura a blocchi / coerenza temporale. Block-structured (BS8/BS16): NON documentato. Sparse GPU operators raggruppano per output-side e input-side sparsity, ma non BS8 invariance. Coerenza temporale maschere: NON misurata. Numeri: nessun numero su overlap temporale. Layer-wise sparsity imbalance: misurato (lower layers più dense, higher layers più sparse), ma è **cross-layer spaziale**, NON temporale.

Regime. Hardware: GPU non specificata (presumibilmente A100/A800 per training; inference su GPU). ProSparse non ha hardware inference specificato nei numeri di speedup. Batch size: non esplicitato; eredita da PowerInfer (batch=1). Memory-bound vs compute-bound: implicito memory-bound (eredita framing).

Cosa copre del nostro caso. ReLUfication post-hoc (customer usa dReLU post-hoc? non chiaro — customer dice *"MLP gated-dReLU ternary"* probabilmente from-scratch); L1 sparsity regularization (customer ha finding *"base ≥ base+regolarizzazione"*); accurate algorithm via sparse operators (predictor-free, affine al customer *"block-structured predictor-free"*); alta sparsità ~89% (vicino al customer 92%). **Cosa NON copre.** Scala 5M (ProSparse è 7B/13B); dReLU (ProSparse usa ReLU/FATReLU); pesi ternari; **block-structured BS8; persistenza temporale**; CPU consumer.

Contraddizioni con le nostre misure. Customer *"predictor falsificati, base ≥ base+reg a op-point parity"*: **[CONTRADDIZIONE POTENZIALE FORTE]**. ProSparse claim che L1 progressive reg è NECESSARIA per spingere 70% (vanilla ReLU) → 89% senza perdita performance (Tab. 1: ProSparse-7B 89.32% sparsity, 38.46 avg vs original-7B 37.96 — match). Customer claim che regolarizzazione NON serve (dReLU da solo dà 92%, base ≥ base+reg). **Possibile spiegazione (NON in paper):** ProSparse parte da SwiGLU LLaMA2 (40% sparsità intrinseca); customer parte da dReLU ternary (92% intrinseca) → regolarizzazione su base già sparsa può non dare ulteriore benefit. **Scale differenti** (7-13B vs 5M) e **architettura** (SwiGLU vs dReLU+ternary) rendono il confronto indiretto. Customer *"block-structured predictor-free BS8, BPB zero"*: ProSparse ha una via *"accurate"* con sparse GPU operators **SENZA predictor** → concettualmente affine al customer

"predictor-free", ma non è BS8-structured e non misura BPB-zero. **[ALLINEATO PARZIALE]** sul principio predictor-free. Customer "persistenza temporale nulla": non misura. **[N/A]**.

1.6 ReLU Strikes Back — Exploiting Activation Sparsity in LLMs

Citazione: [6] Mirzadeh, I.; Alizadeh, K.; Mehta, S.; Del Mundo, C.C.; Tuzel, O.; Samei, G.; Rastegari, M.; Farajtabar, M. (Apple), *ReLU Strikes Back: Exploiting Activation Sparsity in Large Language Models*, ICLR 2024; arXiv:2310.04564. URL: <https://arxiv.org/abs/2310.04564> — slides: <https://imirzadeh.me/publication/relu-act-sparse/iclroralslides.pdf>.

Metodo. Predictor sì/no: **NO, ESPLICITAMENTE.** "Finally, we show that there is no need to train a separate prediction module that complicates the computation graph, and using efficient ReLU layers can be enough." → direttamente predictor-free. Sparsità intrinseca SENZA rete ausiliaria: SÌ. ReLU-induced sparsity 90%+ su OPT (native ReLU). Sparsità sfruttabile via row-skipping hardware-friendly (cuSPARSE, Accelerate su Apple). Activation function: ReLU (substituted da GELU/SiLU) + inserted ReLU (after norm layers) + shifted ReLU (ReLU(x-1)). ReLUfication: SÌ, post-hoc. 2 stage: (1) sostituzione activation function + fine-tuning; (2) inserted ReLU after normalization layers per più sparsità. From-scratch test: OPT 1.3B trained da 0 su 100B RefinedWeb tokens → ReLU match GELU/SiLU performance.

Scala e architettura. Parametri: 1.3B (OPT from scratch), 6.7B (OPT-6.7B, Llama-7B, Falcon-7B finetuned), 7B (Llama-7B, Falcon-7B). Architettura: Transformer decoder-only (OPT, Llama, Falcon — 3 architetture diverse per generalizzabilità). Layer target: FFN/MLP (principale) + Attention (inserted ReLU before QKV/Feedforward, minor cost).

Accuratezza predictor. **N/A** — ReLU Strikes Back non usa predictor.

Costo predictor. Parametri aggiuntivi: 0 (no predictor). Latency overhead: 0 (no predictor). Memory overhead: 0 (no predictor). **[N/A]** — predictor-free by design.

Speedup netto. Speedup algoritmico puro (FLOPS): fino a 3× inferenza FLOPS reduction via inserted ReLU + aggregated sparsity. "We can further reduce inference FLOPS by up to threefold." Speedup wall-clock: NON riportato come numero (paper usa FLOPS come proxy: "we use FLOPS as a proxy for inference efficiency"). **[PROIETTATO]** in wall-clock (paper claim FLOPS proxy is good approximation per structured row-sparsity); **[MISURATO]** in FLOPS. Baseline: dense model (FLOPS).

Struttura a blocchi / coerenza temporale. Block-structured (BS8/BS16): NON documentato. Sparsità è "row-skipping" (structured chunks come righe/columns) — affine al customer "block-structured" ma non numerato come BS8. Coerenza temporale maschere: PARZIALMENTE MISURATA. Concetto di "aggregated sparsity": "ratio of neurons that have not been used up to processing the first t token". Su OPT-6.7B con WikiText prompt: ~50% neuroni inutilizzati nei primi 150 token. Confronto con random: aggregated sparsity > random sparsity → "clear pattern of reusing activations". NON è mask overlap tra token consecutivi, ma cumulative union → metrica correlata ma diversa. **Reuse window test:** $\gamma=16$ — per i token 146-161, riutilizza i pesi dei token 129-145 (no new weight loading). "We observe only a slight increase in perplexity when using this approximation" vs random selection che aumenta PPL significativamente. Numeri: aggregated

sparsity ~50% (OPT-6.7B, 150 token WikiText); random sparsity più bassa (calcolata come s_i^t); $\gamma=16$ con slight PPL increase (NON numerato). **[MISURATO]** su aggregated sparsity e reuse window.

Regime. Hardware: Apple internal GPU (non specificata esplicitamente per inference). Cita "cuSPARSE on NVIDIA CUDA and Accelerate on Apple devices" per row-skipping operators. Batch size: 1 (decoding single-user implicito nel "memory-bound inference step"). Memory-bound vs compute-bound: MEMORY-BOUND esplicito. "This reduction is particularly valuable during the memory-bound inference step, where efficiency is paramount." + "Reduced IO Transfer: weights need to be transferred to the device cache for computation [...] This step constitutes the main bottleneck during token generation".

Cosa copre del nostro caso. COPRE FORTE: predictor-free design (DIRECT MATCH con customer "block-structured predictor-free"); memory-bound framing (forte); ReLUificazione post-hoc; aggregated sparsity / temporal reuse concept (DIRECT MATCH concettuale con customer "persistenza temporale"); 1.3B-7B scale (più vicino al customer 5M rispetto ad altri paper); 3 architetture diverse (generalizzabilità). **Cosa NON copre.** Scala 5M (paper è 1.3B-7B); dReLU (paper usa ReLU/shifted ReLU); pesi ternari; **block-structured BS8 esplicito; persistenza temporale = mask overlap consecutivo** (paper misura aggregated sparsity, metrica diversa); CPU consumer x86 single-thread.

Contraddizioni con le nostre misure. Customer "predictor falsificati": **[ALLINEATO FORTE]**. ReLU Strikes Back esplicitamente: "no need to train a separate prediction module". Customer's finding che predictor non serve è direttamente supportato da questo paper. **Convergenza concettuale forte.** Customer "block-structured predictor-free BS8 18% → 50%, BPB zero": **[ALLINEATO PARZIALE]** — paper è predictor-free, ma NON usa BS8-structured skippable e non misura BPB-zero. Concettualmente affine (row-skipping) ma operativamente diverso. Customer "persistenza temporale nulla": **[CONTRADDIZIONE POTENZIALE]**. ReLU Strikes Back riporta aggregated sparsity NON-RANDOM (real reuse > random reuse) per OPT-6.7B su WikiText. **MA:** aggregated sparsity è "cumulative union di neuroni usati su t token" — diverso da "mask overlap tra token consecutivi". Se customer misura overlap tra token consecutivi (e trova ~0), è compatibile con aggregated sparsity > random: anche con ~0 overlap token-to-token, l'aggregated può crescere lentamente vs random. **Metrica diversa → non contraddizione diretta ma tensione concettuale.** Possibile spiegazione: customer usa dReLU ternary (diverso da OPT ReLU nativo); OPT-6.7B ha 90% sparsity → 10% attivo per token, aggregazione lenta; customer ha 92% hidden → 8% attivo, ma se la selezione è molto più input-specific (dReLU su gate E up vs solo gate), aggregated può essere nullo. **Ipotesi non verificata nel paper.** Reuse window $\gamma=16$ con slight PPL increase: contraddice l'idea "persistenza temporale nulla" — ma è un'approssimazione di inference, non una misura di coerenza naturale delle maschere.

1.7 SparseInfer — Training-free Prediction of Activation Sparsity

Citazione: [7] Shin, J.; Yang, H.; Yi, Y. (Seoul National University), *SparseInfer: Training-free Prediction of Activation Sparsity for Fast LLM Inference*, DAC 2025; arXiv:2411.12692. URL: <https://arxiv.org/abs/2411.12692>.

Metodo. Predictor SÌ, ma **TRAINING-FREE**. Predictor basato su **sign-bit XOR comparison**: paragona il sign bit degli elementi di input X e weight W_{gate} , *predice se l'input a ReLU sarà positivo o negativo senza calcolare l'inner product completo*. "Predicts the sign of each element-wise multiplication result" — usando XOR. Architettura predictor: nessuna rete neurale — solo XOR bitwise + threshold tunabile α (1.00 = neutro, >1.00 = conservativo, <1.00 = aggressivo). Sparsità intrinseca sfruttata parzialmente: sfrutta la sparsità intrinseca di ProSparse-Llama2 (ReLUfied), ma per la selezione dinamica richiede comunque il sign-bit predictor (anche se training-free). Activation function: ReLU (modello target è ProSparse-Llama2, ReLUfied). ReLUfication: N/A (usa ProSparse-Llama2 già ReLUfied).

Scala e architettura. Parametri: 7B, 13B (ProSparse-Llama2-7B, ProSparse-Llama2-13B). Architettura: Transformer decoder-only (LLaMA2 gated MLP). Layer target: MLP/FFN (gate-based MLP).

Accuratezza predictor [MISURATO]. Precision: **>99%** overall, lower nei primi layer (~95-97%). Recall: non riportato come numero singolo; *"relatively lower precision in early layers"*. Baseline: DejaVu/PowerInfer (training-based predictor).

Costo predictor [MISURATO]. Parametri aggiuntivi: 0 (training-free). Memory overhead: 337.5MB vs 1480MB di DejaVu/PowerInfer per Llama2-13B → **4.38× meno memoria**. Solo sign bit packed in 32-bit variables (32 elementi per int). Latency overhead: *"overhead of the predictor was reduced by a factor of four"* compared to the existing activation sparsity prediction method (DejaVu).

Speedup netto [MISURATO], baseline = llama.cpp (no sparsity) + PowerInfer (training-based predictor). Speedup: **1.79× (13B) e 1.74× (7B) vs llama.cpp** a $\alpha=1.00$; **1.27× e 1.30× vs PowerInfer**; ~21% improvement over PowerInfer state-of-the-art. $\alpha=1.02$ (13B): 1.70× vs llama.cpp, 1.21× vs PowerInfer. $\alpha=1.03$ (7B): 1.59× vs llama.cpp, 1.19× vs PowerInfer. Accuracy loss: <1%p su GSM8K/BBH benchmarks. Hardware: NVIDIA Jetson Orin AGX 64GB (mobile-grade GPU, Cortex CPU + Ampere GPU shared LPDDR DRAM).

Struttura a blocchi / coerenza temporale. Block-structured (BS8/BS16): NON documentato. Sign-bit packing è a 32-elementi per int (32-bit variable) — non BS8. Coerenza temporale maschere: NON misurata. Numeri: nessun numero su overlap temporale.

Regime. Hardware: NVIDIA Jetson Orin AGX 64GB (mobile/edge GPU + CPU shared DRAM); CUDA kernel per XOR-based prediction; kernel fusion MLP+activation. Batch size: 1 (decoding single-user, memory-bound esplicitato). Memory-bound vs compute-bound: MEMORY-BOUND esplicito. *"Due to this sequential nature, memory accesses occur frequently in the decoding stage. This can lead to I/O bottlenecks"*.

Cosa copre del nostro caso. Training-free predictor (AFFINE al customer's *"predictor falsificati"* — se anche training-based predictor non serve, training-free è un'alternativa); sign-bit approach (bit-level, affine al customer's ternary pshufb-LUT); memory-bound mobile/edge; <1%p accuracy loss; 4× meno overhead di DejaVu. **Cosa NON copre.** Scala 5M (paper è 7B/13B); dReLU (paper usa ReLU via ProSparse); **pesi ternari**; **block-structured BS8**; **persistenza temporale**; CPU consumer x86 (paper è Jetson Orin ARM+NVIDIA GPU).

Contraddizioni con le nostre misure. Customer "*predictor falsificati*": **[ALLINEATO PARZIALE]**. SparseInfer riconosce che training-based predictor è costoso (memory + retraining per ogni change di quantizzazione/modello) e propone training-free alternative. Customer va oltre: anche training-free predictor non serve, block-structured predictor-free è sufficiente. SparseInfer è un passo intermedio verso customer's position. Customer "*block-structured predictor-free BS8, BPB zero*": SparseInfer non considera BS8 → **[N/A]**. Ma il sign-bit XOR approach è **bit-level**, concettualmente vicino a customer's pshufb-LUT ternary. Customer "*persistenza temporale nulla*": non misura. **[N/A]**. **[OSSERVAZIONE]**: SparseInfer mostra che training-based predictor ha cost elevati (1480MB per Llama2-13B) — coerente con customer's finding che predictor è falsificato (cost > benefit nel regime customer).

2. Domanda 2 — dReLU in letteratura

Tre paper primari identificati: Turbo Sparse (dReLU esplicita, già trattato in 1.4 per il predictor — qui per il confronto qualità vs SiLU/SwiGLU), ReLU Strikes Back (ReLU/shifted ReLU, già trattato in 1.6 — qui per confronto diretto), ProSparse (ReLU/FATReLU, già trattato in 1.5 — qui per confronto). Più menzioni secondarie su GLU family (dReLU è affine al concetto di gated). **Finding chiave: dReLU è usata esplicitamente solo in Turbo Sparse; gli altri paper usano ReLU/shifted ReLU/FATReLU. Nessun paper combina dReLU con pesi ternari.**

2.1 Turbo Sparse — dReLU (estratto focalizzato su qualità)

Citazione: [4] Song, Y. et al. (SJTU-IPADS; Tsinghua), *Turbo Sparse: Achieving LLM SOTA Performance with Minimal Activated Parameters*, arXiv:2406.05955, giugno 2024. URL: <https://arxiv.org/abs/2406.05955>. (Vedi anche sezione 1.4 per il predictor.)

dReLU formula. $\text{Combined_dReLU}(x) := \max(0, x \cdot W_{\text{gate}}) * \max(0, x \cdot W_{\text{up}})$ — ReLU applicato a **ENTRAMBI** gate E up projection. Vs SwiGLU = SiLU(gate) * up. Vs ReGLU = ReLU(gate) * up. **Doppia ReLU** → 90% FFN sparsity (vs 70% di ReGLU). Il vantaggio di dReLU su ReGLU: ReGLU applica ReLU solo al gate, lasciando l'up projection denso → 70% sparsity sul gate ma 0% sull'up → sparsity aggregata ~70%. dReLU applica ReLU a entrambi → sparsity ~90% (combinazione di due sorgenti di sparsità).

Ricetta. ReLUfication post-hoc con continued pretraining. TurboSparse-Mistral-7B: parte da Mistral-7B (SwiGLU), sostituisce con dReLU, continued pretraining su 150B tokens (mixture di dati high-quality). 150B tokens è <1% dei 15T tipici di pretraining LLM → ricetta efficiente. TurboSparse-Mixtral-47B: parte da Mixtral-8x7B (SwiGLU), stessa ricetta. **From-scratch test:** piccolo esperimento a 300M params, 5B tokens, su fineweb → dReLU match SwiGLU performance → conferma che dReLU non perde qualità vs SwiGLU a parità di training da zero.

Costo di qualità misurato [MISURATO], baseline = SwiGLU originale. TurboSparse-Mistral-7B: match performance su benchmark standard (MMLU, commonsense, math) vs Mistral-7B originale. TurboSparse-Mixtral-47B: match performance vs Mixtral-8x7B originale. **BPB:** non riportato (usa accuracy/PPL su benchmark). **From-scratch 300M:** dReLU match SwiGLU

su fineweb validation loss → **costo ≈ 0 a 300M scale**. Sparsità indotta [MISURATO]: dReLU → ~90% FFN sparsity su Mistral-7B (vs 70% ReGLU); 97% su MoE Mixtral-47B (vs 75% base MoE routing).

Confronto col nostro setup. Setup: 5M, dReLU gated MLP ternary, 92% hidden sparse, 79% gate sparse, anchor $+0.013 \pm 0.005$ BPB combinato ternary+dReLU a 5M vs fp32. **[ALLINEATO FORTE]** — TurboSparse riporta 90% FFN sparsity con dReLU a 7B/47B; customer's 92% hidden / 79% gate è coerente (dReLU → ~90% su entrambi i lati). Sul costo di qualità: TurboSparse from-scratch 300M → match SwiGLU (costo ≈ 0 a 300M); customer $+0.013$ BPB a 5M (piccolo ma >0) → consistente con estrapolazione del trend (gap piccolo ma >0 sotto 300M). **Allineato, non contraddittorio.**

Hardware / regime. Training su 64 A800-80G GPUs. Inference: PC consumer (i9-14900HX + RTX 4090, i7-12700K + RTX 2080Ti) + smartphone (OnePlus 12). Batch=1 single-user decoding. Memory-bound esplicito (eredita framing PowerInfer). **Cosa copre.** dReLU activation function (DIRECT MATCH), sparsità intrinseca 90% (DIRECT MATCH), small-scale validation (300M), CPU inference 2-2.8×. **Cosa NON copre.** Scala 5M (min 300M), **pesi ternari** (TurboSparse è fp16/INT4), dReLU+ternary combination, BPB byte-level (usa accuracy). **Contraddizioni:** nessuna diretta. **[ALLINEATO FORTE]** su dReLU + sparsità 90% + costo ≈ 0 a 300M scale.

2.2 ReLU Strikes Back — ReLU/shifted ReLU (confronto diretto)

Citazione: [6] Mirzadeh, I. et al. (Apple), *ReLU Strikes Back: Exploiting Activation Sparsity in Large Language Models*, ICLR 2024; arXiv:2310.04564. URL: <https://arxiv.org/abs/2310.04564>. (Vedi anche sezione 1.6 per il predictor-free design.)

Activation function. ReLU (substituted da GELU/SiLU) + inserted ReLU (after norm layers) + shifted ReLU ($\text{ReLU}(x-1)$). NON usa dReLU esplicitamente. La differenza chiave vs Turbo Sparse: ReLU Strikes Back applica ReLU **solo al gate** ($\text{ReLU}(\text{gate}) * \text{up}$, equivalente a ReGLU) — non all'up projection. Per questo la sparsità è ~85-90% (non ~90% come dReLU). Shifted ReLU ($\text{ReLU}(x-1)$) introduce threshold > 0 per aumentare sparsità al costo di leggera perdita di qualità.

Ricetta. ReLUfication post-hoc. 2 stage: (1) sostituzione activation function (GELU/SiLU → ReLU) + fine-tuning su C4; (2) inserted ReLU after normalization layers per più sparsità. From-scratch test: OPT 1.3B trained da 0 su 100B RefinedWeb tokens → ReLU match GELU/SiLU performance.

Costo di qualità misurato [MISURATO], baseline = GELU/SiLU originale. OPT-1.3B from-scratch: ReLU match GELU performance su validation loss (costo ≈ 0 a 1.3B scale). OPT-6.7B, Llama-7B, Falcon-7B finetuned: ReLU match GELU/SiLU su zero-shot benchmarks (costo $<0.5\%$ accuracy). Shifted ReLU ($\text{ReLU}(x-1)$): leggera perdita (~1-2% accuracy) ma +5-10% sparsità aggiuntiva. **BPB:** non riportato (usa validation loss / zero-shot accuracy). Sparsità indotta [MISURATO]: ReLU → ~85-90% FFN sparsity su OPT; inserted ReLU + shifted ReLU → ~95% aggregated sparsity.

Confronto col nostro setup. Setup: 5M, dReLU gated MLP ternary, 92% hidden sparse, 79% gate sparse, anchor $+0.013 \pm 0.005$ BPB. ReLU Strikes Back usa ReLU/shifted ReLU (non dReLU), ma il principio è affine: ReLU-style activation induce sparsità 85-90%. Costo ≈ 0 a 1.3B scale (from-

scratch); customer +0.013 BPB a 5M → consistente con estrapolazione (gap piccolo ma >0 sotto 1.3B). **[ALLINEATO PARZIALE]** — stessa famiglia concettuale (ReLU-style), ma activation function diversa (ReLU vs dReLU) e sparsità leggermente diversa (85-90% vs 92%). **Non contraddittorio.**

2.3 ProSparse — ReLU + FATReLU (confronto diretto)

Citazione: [5] Song, C. et al. (Tsinghua), *ProSparse: Introducing and Enhancing Intrinsic Activation Sparsity within Large Language Models*, COLING 2025; arXiv:2402.13516. URL: <https://arxiv.org/abs/2402.13516>. (Vedi anche sezione 1.5 per il predictor.)

Activation function. ReLU (sostituita da SwiGLU/SiLU) + FATReLU threshold shifting ($T>0$). NON usa dReLU. La differenza chiave vs Turbo Sparse: ProSparse usa ReLU standard + FATReLU (ReLU con threshold configurabile) per spingere sparsità oltre 70%. $\text{FATReLU}(x, T) = \max(0, x-T)$ con $T>0$ → più aggressivo di ReLU ma meno di dReLU (che applica ReLU a due lati).

Ricetta. ReLUfication post-hoc. 3 step: (1) activation substitution (Swish → ReLU) + continual training; (2) **progressive L1 sparsity regularization** con fattore che aumenta lungo curva sinusoidale in stadi multipli; (3) activation threshold shifting (FATReLU). Innovazione chiave: L1 reg progressiva (vs fissa) → evita shift radicale, mantiene performance.

Costo di qualità misurato [MISURATO], baseline = LLaMA2-7B/13B originale (SwiGLU). ProSparse-7B: 89.32% sparsity, 38.46 avg zero-shot vs original-7B 37.96 — **match performance** (costo ≈ 0 in termini di accuracy). ProSparse-13B: simile match. **BPB:** non riportato (usa zero-shot accuracy). Sparsità indotta [MISURATO]: ReLU vanilla → 70%; ProSparse (ReLU + L1 progressiva + FATReLU) → 89.32%.

Confronto col nostro setup. Setup: 5M, dReLU gated MLP ternary, 92% hidden sparse, 79% gate sparse, anchor +0.013±0.005 BPB. ProSparse usa ReLU + FATReLU (non dReLU), sparsità 89% (vs customer 92%). Costo ≈ 0 a 7B scale (zero-shot match); customer +0.013 BPB a 5M → consistente con estrapolazione. **[ALLINEATO PARZIALE]** — stessa famiglia (ReLU-style), activation diversa (ReLU+FATReLU vs dReLU), sparsità simile (89% vs 92%). **[CONTRADDIZIONE POTENZIALE]** su L1 reg: ProSparse claim che L1 progressiva è necessaria per spingere 70% → 89% senza perdita; customer claim che regolarizzazione NON serve (dReLU da solo dà 92%, $\text{base} \geq \text{base} + \text{reg}$). Possibile spiegazione: ProSparse parte da SwiGLU (40% intrinseca); customer parte da dReLU (92% intrinseca) → reg su base già sparsa può non dare benefit.

2.4 Sintesi dReLU — Mini-tabella comparativa

Mini-tabella dReLU: lavoro $\times$ {scala, ricetta, costo vs baseline, metrica}. Tutti i numeri [MISURATO].

Tabella 2 — dReLU e varianti ReLU-style: scala, ricetta, costo vs baseline, metrica.

Lavoro	Scala	Ricetta	Costo vs baseline	Metrica
Turbo Sparse [4]	300M (from-scratch) + 7B/47B (ReLUfication)	dReLU + continued pretraining 150B tokens (post-hoc) + from-scratch 300M	Match SwiGLU a 300M (costo ≈ 0); match Mistral-7B/Mixtral-47B a 7B/47B (costo ≈ 0 in accuracy)	Validation loss / zero-shot accuracy (no BPB)
ReLU Strikes Back [6]	1.3B (from-scratch) + 6.7B/7B (ReLUfication)	ReLU + inserted ReLU + shifted ReLU; post-hoc 2-stage	Match GELU a 1.3B (costo ≈ 0); match GELU/SiLU a 6.7B/7B (costo $< 0.5\%$ accuracy); shifted ReLU -1-2% accuracy	Validation loss / zero-shot accuracy (no BPB)
ProSparse [5]	7B/13B (ReLUfication)	ReLU + L1 progressiva + FATReLU; post-hoc 3-step	Match LLaMA2 originale a 7B/13B (costo ≈ 0 in accuracy); 89.32% sparsity vs 70% vanilla ReLU	Zero-shot accuracy (no BPB, no PPL)
Customer (anchor)	5M	dReLU gated MLP ternary (presumably from-scratch QAT)	+0.013 $\pm$ 0.005 BPB combinato ternary+dReLU vs fp32 (piccolo ma > 0)	BPB byte-level

Fonti: [4][5][6] + anchor cliente. Tutti i numeri [MISURATO]. Customer anchor: +0.013 $\pm$ 0.005 BPB a 5M.

3. Domanda 3 — Località/caching di esperti MoE

Sette paper identificati + 3 fondativi. Coprono: Mixtral-offloading (LRU + speculative prefetch), Pre-gated MoE (architectural pre-gate), MoE-Infinity (trace-based prediction), Fiddler (popularity static + CPU-GPU orchestration), EdgeMoE (statistical cross-layer prediction), SiDA-MoE (hash + LSTM prediction), Local Routing Consistency (analisi empirica su 20 MoE LLM). **Finding chiave: TUTTI i 7 paper che raggiungono speedup misurabili usano predizione (next-layer, cross-layer, hash+LSTM, trace-based) — NESSUNO si affida a puro working-set locality / LRU.** Local Routing Consistency misura direttamente Mixtral-8x7B SRP@16=49.27% $\approx$ i.i.d. base-rate 50% → ALLINEATO FORTE col nostro finding "working-set $\approx$ i.i.d.".

3.1 Mixtral-offloading — Fast Inference of MoE with Offloading

Citazione: [8] Eliseev, A.; Mazur, D. (MIPT / Yandex SDA / Yandex Research), *Fast Inference of Mixture-of-Experts Language Models with Offloading*, arXiv:2312.17238, dicembre 2023. URL: <https://arxiv.org/abs/2312.17238> — repo: <https://github.com/dvmazur/mixtral-offloading>.

Metodo. Caching esperti SÌ — Sezione 3.1 "Expert Locality and LRU caching": LRU cache di k esperti per MoE layer. Per Mixtral-8x7B: k=2 su 12 GB GPU, k=4 su 16 GB. Offloading SÌ — esperti

split tra host RAM (pinned) e VRAM; buffer contiguo per single host → device copy. 4 buffer on-device per prefetch asincrono. **Prefetching basato su predizione SÌ (SPECULATIVE EXPERT LOADING, Sez. 3.2)** — guess dei prossimi-layer esperti applicando la gating function del *next layer* agli hidden states del *current layer* (sfrutta residualità dei transformer). Carica 1–2 esperti più probabili. Predizione testata 1 / 2 / 10 layer ahead (Fig. 2 destra). LRU cache (no random). Granularità esperti (testata): Mixtral-8x7B (E=8, k=2).

Claim di località/riuso. Claim qualitativo (Sez. 3.1): *"some experts are reused between adjacent tokens"*; *"some experts are active in short sequences of 2-4 tokens, while others are often used with 'gaps'"* (Fig. 1). Misura diretta di "expert reuse rate": PARZIALE — misura *"expert recall"* (cache hit ratio) per diverse k (Fig. 2 sinistra) e *"speculative loading recall"* (Fig. 2 destra) su dataset OpenAssistant. Numeri esatti non estratti dal testo HTML (solo figure; tabella 2 riporta tok/s, non hit-rate). Confronto col base-rate i.i.d.: NON riportato esplicitamente. Il paper non formula mai l'i.i.d. baseline k/E. **[MISURATO]** per hit-rate ma **[PROIETTATO]** per interpretazione i.i.d.

Regime di batch. Single-user (batch=1) esplicito (Sez. 3 e 4.3: *"interactive inference ... batch size 1"*). Generazione token-by-token, OpenAssistant dataset. **Granularità esperti.** E=8, k=2 (Mixtral-8x7B-Instruct). Esperto = FFN Mistral-7B size. 46.7B parametri totali, 45.1B (96.6%) negli esperti.

Hardware. GPU consumer / free-tier cloud: T4 16 GB (Colab, PCIe Gen.3), RTX 3060 12 GB (PCIe Gen.3), RTX 3080 Mobile 16 GB (PCIe Gen.4), A100-80GB-SXM (riferimento datacenter). DRAM host: sufficiente a tenere tutto il modello (tranne Colab per 3-bit). Hot-pool size = $k \times \text{num_layers}$ esperti (k=2 o 4 per layer).

Speedup misurato [MISURATO], baseline = HuggingFace accelerate. Tabella 2 (Mixtral-8x7B, tok/s, batch=1, 2-bit/3-bit esperti): Full algorithm (LRU + speculative pre-loading): 2.09–3.06 tok/s (T4 1.6–2.04; 3060 2.04–2.28; 3080 Mobile 2.48–2.66; A100 2.85–3.06). W/o expert pre-loading (solo LRU): 1.37–2.68 tok/s. W/o LRU cache & pre-loading (naive offloading): 1.06–2.27 tok/s. Naive offloading (accelerate baseline): 0.58–1.39 tok/s. Speedup $\approx 2\times$ vs naive accelerate; $\approx 1.2\text{--}1.5\times$ vs LRU-only.

Memory bound o compute bound. Memory-bound dichiarato esplicitamente (Sez. 2.3): *"interactive inference ... spends most of the time waiting for next layer's parameters to be loaded"*. *"both GPUs are still bottlenecked by host-to-device bandwidth, limited by the PCIe architecture"*.

Cosa copre del nostro caso. Batch=1 single-user, GPU consumer + DRAM offload, regime memory-bound, uso di LRU + speculative prediction (entrambi i meccanismi che diciamo necessari). Mixtral 8x7B è architettura di reference (k/E = 25%, identico al nostro). **Cosa NON copre.** Granularità diversa (E=8/k=2 coarse vs nostro E=32/k=8 granular); nessun test su CPU consumer x86; nessun confronto esplicito con base-rate i.i.d.; LRU è il meccanismo principale (noi diciamo hot-pool falsificato → LRU da solo non basta); modello target è Mixtral non granular-MoE.

Contraddizioni con le nostre misure. [NEUTRO / ALLINEATO SUL MECCANISMO] — Il paper osserva reuse *"in short sequences of 2-4 tokens"* (claim debole, qualitativo). NON misura esplicitamente se questa persistenza > base-rate k/E. Il fatto che il paper debba aggiungere *"speculative prediction"* oltre a LRU è COERENTE con il nostro finding *"località solo da*

predizione": se LRU bastasse, non servirebbe speculative loading. **[ALLINEATO]** sul fatto che LRU da solo è insufficiente e la predizione è necessaria.

3.2 Pre-gated MoE — Algorithm-System Co-Design

Citazione: [9] Hwang, R. (KAIST); Wei, J. (USTC/MSR); Cao, S.; Hwang, C.; Tang, X.; Cao, T.; Yang, M. (Microsoft Research), *Pre-gated MoE: An Algorithm-System Co-Design for Fast and Scalable Mixture-of-Expert Inference*, ISCA 2024; arXiv:2308.12066. URL: <https://arxiv.org/abs/2308.12066> — repo: https://github.com/ranggihwang/Pregated_MoE.

Metodo. Offloading SÌ — esperti in CPU memory, subset migrato a GPU on-demand. Caching esperti / hot-pool SÌ implicito — *"expert buffering is a technique that can help reduce inference latency by caching hot, active experts in GPU memory"*. **Prefetching basato su predizione SÌ, MA ARCHITETTONICO** — invece di predire esperti del prossimo layer (come Mixtral-offloading), **modifica l'architettura MoE** introducendo una **pre-gate function** nel blocco N che seleziona gli esperti del blocco N+1 (non c'è più sequential dependency tra selection ed execution nello stesso blocco). Sistema: **preemptive expert migration** sovrappone CPU → GPU transfer con expert execution del blocco corrente. Baselines: MoE-OnDemand (on-demand), MoE-Prefetch (SE-MoE prefetch-all), oracular GPU-only (riferimento upper bound). Granularità: si riferisce a SwitchTransformer (E=128) come esempio; test specifici su modelli non estratti in dettaglio dal testo HTML.

Claim di località/riuso. NON misura direttamente expert reuse rate. Il paper non assume località nativa forte — al contrario, **modifica il modello** per **crearla** artificialmente. Confronto col base-rate i.i.d.: NON riportato. **[PROIETTATO]** (il design presuppone che località nativa non basti, e per questo serve pre-gating).

Regime di batch. Batch=1 esplicito (Sez. VI): *"real-world production ML serving systems are optimized for a batch size of 1"*. Single MoE block execution comparison. **Granularità esperti.** Setup testati: riferiti a SwitchTransformer (E=128, k=1) come esempio. Dettagli tabella non estratti (figure-heavy). h non riportato esplicitamente.

Hardware. GPU datacenter: AMD EPYC 7V12 64-Core CPU + 1.8 TB DDR4 + NVIDIA A100 80 GB HBM, PCIe Gen4 32 GB/s.

Speedup misurato [MISURATO], baseline = oracular GPU-only + MoE-OnDemand + MoE-Prefetch. *"only adding 23% performance overhead than the oracular, performance-optimal GPU-only solution"*. **GPU memory peak ridotto 4.2× vs GPU-only.** vs MoE-OnDemand e MoE-Prefetch: Pre-gated MoE migliore (riduce serialization overhead), ma numeri esatti di speedup end-to-end non estratti dal testo HTML. Memory-bound per design — il sistema nasce per *"tackle the compute and memory challenges"*. CPU → GPU migration latency è il bottleneck esplicito.

Cosa copre del nostro caso. Batch=1, regime memory-bound, offload CPU → GPU, uso di prediction per prefetch (analogamente al nostro *"località solo da predizione"*). **Cosa NON copre.** Richiede **re-training** dell'architettura MoE (pre-gate function) → non applicabile a modelli off-the-shelf; hardware datacenter A100 (non consumer x86); non testa granular MoE E32/k8; noi non possiamo ri-addestrare il modello.

Contraddizioni con le nostre misure. [ALLINEATO FORTE] — Il paper IMPLICITAMENTE conferma che la località nativa di MoE è debole (altrimenti non servirebbe pre-gating architetturale). La necessità di *intervenire algoritmico-architetturale* per creare prefetchable locality è la versione forte del nostro finding "*località solo da predizione*". Se il working-set fosse fortemente locale di per sé, Pre-gated MoE non aggiungerebbe valore.

3.3 MoE-Infinity — Sparsity-Aware Expert Cache on Personal Machines

Citazione: [10] Xue, L.; Fu, Y.; Lu, Z.; Sun, C.; Mai, L.; Marina, M. (University of Edinburgh), *MoE-Infinity: Efficient MoE Inference on Personal Machines with Sparsity-Aware Expert Cache*, ICML 2024; arXiv:2401.14361. URL: <https://arxiv.org/abs/2401.14361> — repo: <https://github.com/EfficientMoE/MoE-Infinity>.

Metodo. Caching esperti / hot-pool SÌ — Sparsity-Aware Expert Cache (PEC). Trace-based: traccia Expert Activation Matrix (EAM) a livello iteration (iEAM) e request (rEAM). Offloading SÌ — model residente in host memory, activated experts fetched in GPU. **Prefetching basato su predizione SÌ (trace-based).** Predice attivazione futura confrontando pattern osservati con gruppi di attivazioni storiche (K-means clustering). Sostituisce LRU/LFU (insufficienti per MoE). LRU esplicitamente scartato — Sez. 4.2: LRU/LFU non catturano sparse activation patterns intra-request; sostituito da trace prediction. Granularità testata: DeepSeek-V2-Lite (E=64, k=6, 2 shared), Mixtral-8x7B (E=8, k=2), Switch-MoE (E=128, k=1), NLLB-MoE (E=128, k=2), Arctic-MoE (E=128 small experts, 900 GB).

Claim di località/riuso [MISURATO FORTE]. Claim quantitativo (Sez. 4.1): "*For MoE models with around 100 experts (e.g., DeepSeek, QWen-MoE, NLLB, and Switch-MoE), fewer than 5% of experts are repeatedly activated when decoding tokens for a single request. Even for MoE models with fewer experts (e.g., Mixtral), we observe only 25% activation per request.*" Per Mixtral (E=8, k=2): $25\% = 2/8 = k/E$ — coincide con base-rate i.i.d. → **AMBIGUO:** o significa "2 esperti sempre gli stessi in tutta la request" (forte località) oppure "*ratio attivi/totali = 25%*" (banale = k/E). Per DeepSeek (E=64, k=6): $5\% = 3.2/64 \approx 3$ esperti. Sotto $k=6$ → claim di working-set $< k$ → forte località. Confronto col base-rate i.i.d.: NON esplicito, ma implicito nel framing "*skewed vs uniform*". Claim di confine (Contribution 1): "*this reuse pattern is only present at the request level; after processing multiple requests, the skew disappears, and all experts tend to be activated uniformly.*" Figura 2: Expert reuse count per request, Mixtral-8x7B last layer — Sequence 1: Expert 2 reused >15 volte (7× più di qualunque altro). Sequence 2: Expert 0 e 5 dominano. → skewed intra-request.

Regime di batch. Single-user, batch=1 esplicito (Sez. 1): "*personal machines ... single-user environments ... batch size of one*". Prompt 512 token, decode 32 token. **Granularità esperti.** Modelli testati: DeepSeek-V2-Lite (E=64, k=6, 2 shared), Mixtral-8x7B (E=8, k=2), Switch (E=128, k=1), NLLB-MoE (E=128, k=2), Arctic (900 GB).

Hardware. GPU consumer: NVIDIA A5000 24 GB, PCIe 4.0 (24 GB/s). DRAM host sufficiente per modelli 100 GB+.

Speedup misurato [MISURATO], baseline = DeepSpeed + Mixtral-Offloading + Ollama + vLLM + BrainStorm. Tabella 1 (DeepSeek-V2-Lite, A5000-24GB, batch=1): GPU idle time /

TPOT avg / TPOT tail. DeepSpeed: 513 ms / 737 ms / 803 ms. Mixtral-Offloading: 754 ms / 1250 ms / 1530 ms. Ollama: 2073 ms / 2590 ms / 2599 ms. vLLM: 254 ms / 485 ms / 493 ms. **MoE-Infinity: 51 ms / 155 ms / 169 ms.** 3.1–16.7× TPOT improvement vs vLLM/Ollama/DeepSpeed/Mixtral-Offloading/BrainStorm. Worst-case su Mixtral (E=8/k=2, small E + high activation ratio): solo 1.4× su BrainStorm/Mixtral-Offloading.

Memory bound o compute bound. Memory-bound esplicito: *"substantial I/O bottlenecks on the PCIe bus", "high GPU idle time", "inaccurate predictions can degrade performance even further than on-demand fetching approaches like vLLM"*.

Cosa copre del nostro caso. Batch=1 single-user, GPU consumer (A5000 24GB), memory-bound, offload DRAM, usa prediction (trace-based, non LRU). Testato su DeepSeek-V2-Lite (E=64, k=6, granular MoE affine al nostro E32/k8). **Cosa NON copre.** Hardware A5000 non x86 consumer; usa modelli pretrained (no E32/h128 specifico); il claim *"25% per request"* per Mixtral è ambiguo.

Contraddizioni con le nostre misure. [ALLINEATO cross-request, CONTRADDIZIONE PARZIALE intra-request]. Cross-request [ALLINEATO FORTE]: *"after processing multiple requests, the skew disappears, and all experts tend to be activated uniformly"* → coincide esattamente col nostro finding working-set $\approx$ i.i.d. (in regime multi-request / i.i.d. workload). Intra-request [CONTRADDIZIONE PARZIALE]: MoE-Infinity claim skewed reuse intra-request (Fig. 2). Noi diciamo working-set $\approx$ i.i.d. → se la nostra misura è intra-request, c'è contraddizione. **Caveat:** la loro misura è su workload specifico (Mixtral 8x7B su BigBench/MMLU/Flan); il nostro E32 granular potrebbe avere skewed reuse anche più debole (vedi Local Routing Consistency, 3.7, che mostra granular MoE = località più bassa).

3.4 Fiddler — CPU-GPU Orchestration for Fast MoE Inference

Citazione: [11] Kamahori, K.; Tang, T.; Gu, Y.; Zhu, K.; Kasikci, B. (University of Washington; Tsinghua), *Fiddler: CPU-GPU Orchestration for Fast Inference of Mixture-of-Experts Models*, ICLR 2025; arXiv:2402.07033. URL: <https://arxiv.org/abs/2402.07033> — repo: <https://github.com/efeslab/fiddler>.

Metodo. Caching esperti / hot-pool SÌ — offline popularity profiling. *"place frequently-used experts on the GPU based on offline profiling of expert popularity"*. Offloading SÌ — model weights in CPU memory, transferred to GPU on-demand. **Prefetching basato su predizione NO — orchestrato staticamente.** Fiddler sviluppa un latency model basato su batch size: per input piccoli → esegue su CPU (evita transfer overhead); per input grandi → trasferisce a GPU. CPU kernel custom AVX512_BF16. LRU NON usato. Strategia = popularity statica + CPU-GPU orchestration. Granularità testata: Mixtral-8x7B (E=8, k=2) 16-bit, 47B params, ~90 GB.

Claim di località/riuso. NON misura expert reuse rate diretto. Popularity = frequenza media di attivazione su dataset (ShareGPT), non skewed intra-request. Confronto col base-rate i.i.d.: NON riportato. [PROIETTATO].

Regime di batch. Tre scenari espliciti: single-batch inference (batch=1), long-prefill processing (batch grande di prompt), beam search (batch=N per beam). Workload: ShareGPT subset.

Granularità esperti. Mixtral-8x7B: $E=8, k=2$. Su $32 \text{ layer} \times 8 \text{ esperti} = 256 \text{ esperti totali}$. Memory budget: 56–125 esperti su GPU dei 256 (Tab. 1).

Hardware. GPU consumer/workstation: Environment 1 = Quadro RTX 6000 24 GB (PCIe Gen3 x16, 32 GB/s) + Intel Xeon Gold 6126 48-core. Environment 2 = RTX 6000 Ada 49 GB (PCIe Gen4 x16, 64 GB/s) + Intel Xeon Platinum 8480+ 112-core.

Speedup misurato [MISURATO], baseline = Mixtral-Offloading + DeepSpeed-MII + llama.cpp. $1.26\times$ single-batch inference; $1.30\times$ long-prefill processing; $11.57\times$ beam search inference. Memory bound (model transfer) per single-batch; compute-bound (CPU) per large-batch quando CPU è troppo lento. Fiddler fa switching dinamico.

Cosa copre del nostro caso. GPU consumer/workstation + CPU + DRAM offload, batch=1 single-user testato, modello Mixtral ($E=8/k=2$, stesso base-rate 25% del nostro). **Cosa NON copre.** Granularità $E=8$ (non $E=32$); popularity è statica cross-request (noi diciamo hot-pool falsificato in regime i.i.d. → popularity-based è coerente con fallimento hot-pool); non testa E32/h128; CPU kernel usa AVX512_BF16 (non x86 legacy come Zen 2).

Contraddizioni con le nostre misure. [NEUTRO] — Fiddler usa popularity profiling offline (cross-request aggregate), non intra-request working-set. Implicitamente riconosce che la località intra-request non è sfruttabile staticamente → si appoggia a CPU-GPU orchestration dinamica. **[ALLINEATO indiretto]** sul fatto che hot-pool statico non basta in regime i.i.d. (il loro popularity profile è aggregato, non working-set intra-request).

3.5 EdgeMoE — Fast On-Device Inference of MoE-based LLMs

Citazione: [12] Yi, R.; Guo, L. (Huawei); Wei, S.; Zhou, A.; Wang, S.; Xu, M. (BUPT), *EdgeMoE: Fast On-Device Inference of MoE-based Large Language Models*, IEEE TMC 2025; arXiv:2308.14352. URL: <https://arxiv.org/abs/2308.14352>.

Metodo. Caching esperti / hot-pool SÌ — expert buffer in memory size M ; cache eviction policy basata su frequency + layer index (non puro LRU). Offloading SÌ — esperti su storage esterno (SSD/HDD); non-expert weights in device memory. **Prefetching basato su predizione SÌ (statistical profile inter-layer)** — sfrutta correlazione: $P(E_n | E_0..E_{n-1})$ — *predice esperti del layer n dal routing dei layer precedenti*. ****Offline phase**:** *costruisce dizionario {activation status of experts from 2 consecutive MoE layers → probabilities of each expert in next layer}*. ****Online phase**:** *query del dizionario + preload 1–3 esperti più probabili*. Esempio (Fig. 8): ST-base-8 su SAMsum, $P(E_3=5 | E_1=3, E_2=1) = 87.1\%$. LRU No, eviction custom (frequency + layer proximity). Granularità testata: ST-base-8 ($E=8, k=1$), ST-base-16/32/64/128 ($k=1$), ST-large-128 ($E=128, k=1, 26B$), GPTSAN ($E=16, k=2$). 0.4B–26B params.

Claim di località/riuso. NON claim di working-set locality intra-request. Claim di **correlazione inter-layer** (87.1% prediction confidence tra layer consecutivi). Confronto col base-rate i.i.d.: NON riportato. Fig. 13: cache hit ratio per 5 eviction policy (LRU/LFU/FIFO + loro) al variare di expert buffer size — loro policy vince. **[MISURATO]** per predizione inter-layer; **[PROIETTATO]** per working-set locality.

Regime di batch. Single-user on-device inference. Per-token latency misurata. **Granularità esperti.** Switch Transformer base $E=8/16/32/64/128$, $k=1$. ST-large-128 = 26B. GPTSAN $E=16$, $k=2$.

Hardware. Edge devices: NVIDIA Jetson TX2 (embedded GPU + ARM), Raspberry Pi 4B (ARM CPU only). Confronto SSD vs HDD.

Speedup misurato [MISURATO], baseline = IO-FREE (no offload) + IO-EXP (on-demand) + IO-QEXP (quantized on-demand) + STI (static). Jetson TX2 SSD: $2.63\text{--}3.01\times$ vs IO-EXP. Raspberry Pi 4B: $4.49\text{--}5.43\times$ vs IO-EXP. $1.35\text{--}1.5\times$ vs IO-QEXP, $1.11\text{--}2.78\times$ vs STI. Memory savings $2.6\text{--}3.2\times$ vs IO-FREE. Memory-bound esplicito (expert I/O swapping è il bottleneck). Su HDD: $4.49\text{--}4.76\times$ vs $2.63\text{--}3.01\times$ su SSD → bottleneck storage read.

Cosa copre del nostro caso. On-device single-user, memory-bound, offload esterno (storage/DRAM), uso di prediction (inter-layer, non working-set). **Cosa NON copre.** Hardware edge ARM (non x86 consumer); modelli Switch Transformer ($k=1$, molto sparse — diverso da $E=32/k=8$); granularity differente; prediction basata su statistical profile offline (non online learning).

Contraddizioni con le nostre misure. [ALLINEATO] — EdgeMoE usa predizione inter-layer ($P(E_n | E_{0..E_{n-1}})$) per prefetch, non working-set locality intra-request. Il fatto che la predizione raggiunga 87.1% su ST-base-8 conferma che la località **esiste tra layer** (cross-layer prediction possibile), non tra token consecutivi nello stesso layer. **Coerente col nostro finding "località solo da predizione"** — ma qui la predizione è cross-layer, non next-token.

3.6 SiDA-MoE — Sparsity-Inspired Data-Aware Serving

Citazione: [13] Du, Z.; Li, S.; Wu, Y.; Jiang, X.; Sun, J.; Zheng, Q.; Wu, Y.; Li, A.; Li, H.; Chen, Y. (Duke University et al.), *SiDA-MoE: Sparsity-Inspired Data-Aware Serving for Efficient and Scalable Large Mixture-of-Experts Models*, MLSys 2024; arXiv:2310.18859. URL: <https://arxiv.org/abs/2310.18859> — repo: <https://github.com/timlee0212/SiDA-MoE>.

Metodo. Caching esperti / hot-pool SÌ — data-aware expert activation prediction. Esperti in main memory (CPU RAM), subset in GPU memory. Offloading SÌ. **Prefetching basato su predizione SÌ (hash function + LSTM with sparse attention)** — predice top-1 (SST2) o top-3 (MRPC/MultiRC) esperti. "Hash hits rate" = prediction accuracy. LRU No — hash-based prediction. Granularità testata: Switch-base-8 ($E=8$, $k=1$), Switch-base-64, Switch-base-128, Switch-base-256 ($k=1$).

Claim di località/riuso. NON misura working-set locality diretta. Misura "hash hits rate" = prediction accuracy della hash function. Claim indiretto (Sez. 2.4): "Sparse Activation of Experts in Large MoE Models" — sfrutta sparsità, non località. Confronto col base-rate i.i.d.: NON riportato. [PROIETTATO].

Regime di batch. Batch=1 esplicito (Sez. 4 Setup): "We set the batch size as 1 when measuring the latency and memory usage to eliminate the disturbance of the batch size." **Granularità esperti.** Switch-base-8/64/128/256, $k=1$ (top-1 routing). Models from 0.22B (Switch-base-8) to 7.1B (Switch-base-128).

Hardware. GPU datacenter: NVIDIA A100 80 GB + $64\times$ Intel Xeon Platinum 8358 @ 2.60 GHz. **Speedup misurato [MISURATO], baseline = Standard + DeepSpeed + Tutel.** Throughput: $2.60\times$

(Switch-base-128) / $3.93\times$ (Switch-base-256) su SST2; $2.52\times$ / $3.83\times$ su MRPC; $1.26\times$ / $1.57\times$ su MultiRC. Latency ridotta fino a 28% (i.e. $3.6\times$ riduzione) su Switch-base-256. GPU memory saving: 80% su SST2, 60–80% su MRPC, 20–40% su MultiRC. Performance drop < 1%. Memory-bound dichiarato.

Cosa copre del nostro caso. Batch=1, memory-bound, offload CPU → GPU, prediction-based prefetch (hash + LSTM). **Cosa NON copre.** Hardware datacenter A100 (non consumer x86); modelli Switch Transformer k=1 (molto diversi da E=32/k=8); non testa granular MoE; usa hash function trained offline (data-aware ma non working-set locality).

Contraddizioni con le nostre misure. [NEUTRO] — SiDA-MoE usa prediction esperta basata su hash + LSTM, non assume working-set locality. Implicitamente **[ALLINEATO]** sul fatto che prediction è necessaria. Non misura base-rate i.i.d.

3.7 Local Routing Consistency — Analisi empirica su 20 MoE LLM (KEY)

Citazione: [14] Liang, J. (Fudan); Wang, S. (USC); Tian, M.; Li, Y.; Tang, D. (Huawei); Wei, Z. (Fudan), *Not All Models Suit Expert Offloading: On Local Routing Consistency of Mixture-of-Expert Models*, arXiv:2505.16056, 2025. URL: <https://arxiv.org/abs/2505.16056> — repo: <https://github.com/ljcleo/moe-lrc>.

Metodo. NON è un sistema di caching — è uno **studio empirico** di località su 20 MoE LLM. Definisce 2 metriche: **SRP (Segment Routing Best Performance)**: quanto bene un segment router (fixed group di esperti per m token consecutivi) approssima le decisioni del router originale. Senza parametri tranne segment length m. **SCH (Segment Cache Best Hit Rate)**: hit rate ottimale con cache size limit pk. Modelli testati (Tab. 4): 20 MoE LLM da 3B a 57B — PowerMoE-3B (E=40, k=8), LLaMA-MoE-v1 (E=16, k=4), OLMoE (E=64, k=8), SwitchTransformers-Base-128 (E=128, k=1), LLaMA-MoE-v2 (E=8, k=2), JetMoE (E=8, k=2), OpenMoE (E=32, k=2, +1 shared), MiniCPM-MoE (E=8, k=2), Qwen1.5-MoE (E=60, k=4, +4 shared), DeepSeek-V2-Lite (E=64, k=6, +2 shared), **DeepSeekMoE (E=64, k=6, +2 shared)**, XVERSE-MoE (E=64, k=6, +2 shared), Qwen3-30B (E=128, k=8), Yuan2.0-M32 (E=32, k=2), Phi-3.5-MoE (E=16, k=2), GRIN-MoE (E=16, k=2), **Mixtral-8x7B (E=8, k=2)**, Jamba-Mini-1.6 (E=16, k=2, hybrid SSM-Transformer), NLLB-MoE-54B (E=128, k=2), Qwen2-57B (E=64, k=8, +8 shared). Dataset: RedPajama 7 domini (CC, Books, Wikipedia, ArXiv, StackExchange, GitHub), 2048 seq × 512 token per dominio = 14.336 input samples.

Claim di località/riuso [MISURATO DIRETTAMENTE — gold standard]. SRP@16 per modelli chiave (Tab. 1): LLaMA-MoE-v2: 79.59% (più alta). Yuan2.0: 63.75%. Qwen3: 55.90%. **PowerMoE (E=40, k=8): 54.69%** ← setup più vicino al nostro E=32/k=8. Phi-3.5-MoE: 51.59%. GRIN-MoE: 50.55%. **OLMoE (E=64, k=8): 50.17%** ← molto vicino al nostro k=8. **Mixtral-8x7B: 49.27%**. MiniCPM-MoE: 48.93%. JetMoE: 47.46%. LLaMA-MoE-v1: 45.28%. XVERSE-MoE (granular + shared): 38.03%. Jamba-Mini-1.6: 37.91%. DeepSeek-V2-Lite (granular + 2 shared): 37.79%. **DeepSeekMoE (granular E=64/k=6 + 2 shared): 36.79%** ← setup granular affine al

nostro. NLLB-MoE: 24.84–30.94%. Qwen1.5-MoE (with shared): 30.88%. SwitchTransformers (E=128, k=1): 18.98–18.90% (più bassa).

Confronto col base-rate i.i.d. ESPLETATO (calcolo nostro, non del paper). Per cache size pk (con $p=2$) e i.i.d. uniform gating: base-rate hit $\approx pk/E$. **Mixtral-8x7B (E=8, k=2, $p=2 \rightarrow$ cache=4): i.i.d. base-rate $\approx 4/8 = 50\%$.** SRP@16 misurato = 49.27% $\rightarrow$ **praticamente ALLINEATO a i.i.d.** (gap -0.7 pp). **DeepSeekMoE (E=64, k=6, $p=2 \rightarrow$ cache=12): i.i.d. base-rate $\approx 12/64 = 18.75\%$.** SRP@16 misurato = 36.79% $\rightarrow$ +18 pp sopra i.i.d. (alcuna località). **PowerMoE (E=40, k=8, $p=2 \rightarrow$ cache=16): i.i.d. base-rate $\approx 16/40 = 40\%$.** SRP@16 misurato = 54.69% $\rightarrow$ +14.7 pp sopra i.i.d.

SCH a $p=2$ per Mixtral-8x7B (Tab. 5, 95% CI). $m=4$ (segmento 4 token): **88.16–88.24%** (vs i.i.d. 50%, +38 pp — località forte a corto raggio). $m=16$: **73.75–73.87%** (vs 50%, +24 pp). $m=64$: **65.91–66.07%** (vs 50%, +16 pp). $m=256$: **61.75–61.95%** (vs 50%, +12 pp) $\rightarrow$ **convergenza verso i.i.d.** al crescere di m . Claim chiave (Sez. 3.2): *"There is a gap between short-term ($m=4$) and long-term ($m \geq 16$) local routing consistency, where many models exhibit the short-term one but only a few demonstrate the long-term one."* Claim architetturale (Sez. 3.3): *"Apply MoE on every layer + no shared experts $\rightarrow$ highest local routing consistency."* Modelli granular con shared experts (DeepSeek, XVERSE) hanno località PIÙ BASSA. SCH ottimale (Sez. 5.1): *"cache sizes approximately $2 \times$ the active experts" ($p \approx 2$) è il knee-point.*

Regime. Non inference serving — analisi offline di trace di routing su dataset. Equivalenti a single-request (sequence-by-sequence). $m=4/16/64/256$ token segmenti. **Granularità esperti.** 20 modelli, E da 8 (Mixtral) a 128 (Switch/Qwen3), k da 1 a 8, h non riportato uniformemente. Setup più vicino al nostro: PowerMoE-3B (E=40, k=8), OLMoE (E=64, k=8), Qwen2-57B (E=64, k=8 + 8 shared). **Hardware.** Non inference — analisi di trace di routing su modelli pretrained. Hardware non rilevante (si misura qualità di routing, non throughput). **Speedup:** N/A — il paper non misura speedup, solo metriche di routing locality (SRP, SCH).

Cosa copre del nostro caso. COPRE FORTEMENTE: è l'unico paper che misura direttamente località di routing su un setup granular affine al nostro (E=40–64, k=8 $\rightarrow$ PowerMoE/OLMoE). Quantifica esattamente il gap sopra i.i.d. a diverse scale di segmento. **Cosa NON copre.** Non usa esattamente E=32/h128/top-8; non è un sistema (non misura speedup); non testa single-user bandwidth-bound.

Contraddizioni con le nostre misure. [ALLINEATO FORTE E QUANTITATIVO]. Mixtral-8x7B (E=8, k=2, base-rate 25%): SRP@16=49.27% $\approx$ i.i.d. base-rate 50% $\rightarrow$ **ALLINEATO** con nostro finding working-set $\approx$ i.i.d. (gap -0.7 pp, essenzialmente i.i.d.). Mixtral-8x7B SCH@256, $p=2$ = 61.85% vs i.i.d. base-rate 50% $\rightarrow$ **località marginale** (+12 pp). Allineato con *"località solo da predizione"* — anche con cache ottima $2 \times k$, l'hit-rate a lungo range supera i.i.d. di solo 12 pp. PowerMoE (E=40, k=8, setup più vicino al nostro E=32/k=8): SRP@16=54.69% vs i.i.d. 40% $\rightarrow$ +14.7 pp (località marginale, sempre più vicina a i.i.d. che a *"hot-pool concentrato"*). DeepSeekMoE granular (E=64, k=6 + 2 shared): SRP@16=36.79% $\rightarrow$ +18 pp sopra i.i.d. base-rate 18.75%. Modelli granular con shared hanno località PIÙ BASSA dei modelli coarse senza shared. Trend generale: località $\rightarrow$ converge a i.i.d. al crescere di m (segmento). Short-range ($m=4$) mostra

località +38 pp; long-range (m=256) solo +12 pp. **Coerente col nostro finding** se interpretiamo "working-set $\approx$ i.i.d." come misura su segmenti lunghi o multi-request.

3.8 Fondativi (Switch Transformer, DeepSeekMoE, Lina)

Switch Transformer [15] Fedus, W.; Zoph, B.; Shazeer, N. (Google), *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*, JMLR 2022; arXiv:2101.03961. k=1 (top-1 routing), E=128 (fino a 2048 nel largest). **Auxiliary load balancing loss** (Eq. 4) forza uniform routing $\rightarrow$ working-set $\approx$ i.i.d. BY DESIGN. **[ALLINEATO FORTE]** — spiega *perché* la maggior parte dei MoE moderni hanno working-set $\approx$ i.i.d.: il load balancing loss è standard practice. Il finding del cliente è conseguenza diretta del design di Switch Transformer e successori.

DeepSeekMoE [16] Dai, D. et al. (DeepSeek-AI), *DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models*, ACL 2024; arXiv:2401.06066. Architettura granular: mN esperti (m $\times$ più piccoli) con mK attivi + K_s shared. Es. E=64, k=6, 2 shared (m=4 da N=16/K=2 base). Claim: finer expert segmentation $\rightarrow$ migliore specialization $\rightarrow$ migliore qualità a parità di FLOPs. NON parla di caching/locality (paper architetturale, non system). Però Local Routing Consistency mostra che questa architettura ha località PIÙ BASSA dei MoE coarse (SRP@16=36.79% per DeepSeekMoE vs 49.27% per Mixtral).

Lina [17] Li, Y. et al., *Lina: Accelerating Relational Alignment in Multi-Agent Large Language Model Systems*, USENIX ATC 2023; arXiv:2210.17223. Distribuito/multi-GPU inference (16 $\times$ A100 40GB + 100 Gbps InfiniBand), non single-user. Popularity-based scheduling: stima expert popularity cross-layer da trace precedenti. Speedup: 2.21 $\times$ all-to-all, 1.57 $\times$ training step, 1.45 $\times$ median inference / 1.63 $\times$ p95 inference vs DeepSpeed. **NON COPRE il nostro caso** (no single-user consumer, no offload DRAM).

3.9 Sintesi MoE — Convergenze e divergenze

Convergenza forte della letteratura su prediction-based prefetch. TUTTI i 7 paper che raggiungono speedup misurabili (Mixtral-offloading, Pre-gated MoE, MoE-Infinity, EdgeMoE, SiDA-MoE, Fiddler indirettamente, Local Routing Consistency come analisi) usano **predizione** (next-layer, cross-layer statistical, hash+LSTM, o trace-based) — NESSUNO si affida a puro working-set locality / LRU. **Questo VALIDA direttamente il nostro finding "località solo da predizione"**.

Spiegazione architetturale del finding _working-set $\approx$ i.i.d._. Switch Transformer (Fedus 2022) adotta auxiliary load balancing loss che forza uniform routing $\rightarrow$ working-set $\approx$ i.i.d. BY DESIGN. La maggior parte dei MoE moderni eredita questa pratica $\rightarrow$ il finding del cliente è conseguenza architetturale, non anomalia.

Granularità e shared experts riducono località. Local Routing Consistency mostra DeepSeekMoE (granular + 2 shared) SRP@16=36.79% < Mixtral-8x7B (coarse, no shared) 49.27%. Il nostro setup E32 granular è più affine a DeepSeekMoE che a Mixtral $\rightarrow$ aspettiamo località ANCOR PIÙ VICINA a i.i.d. rispetto alla letteratura su Mixtral.

Gap di evidenza diretta. NESSUN paper testa esattamente $E=32$, $h=128$, top-8, su consumer x86 CPU+GPU con esperti streamed da DRAM. Setup più vicini: MoE-Infinity (DeepSeek-V2-Lite $E=64/k=6$ su A5000 consumer), Local Routing Consistency (PowerMoE $E=40/k=8$ e OLMoE $E=64/k=8$, analisi tracce).

Speedup realistici nel regime cliente (consumer GPU + DRAM offload + batch=1): $2\times$ (Mixtral-offloading vs naive), $1.26\times$ (Fiddler vs Mixtral-offloading), $3.1\text{--}16.7\times$ (MoE-Infinity vs vLLM/DeepSpeed). Tutti memory-bound, tutti con prediction come meccanismo chiave.

4. Tabella riassuntiva lavoro $\times$ dimensioni (Q1 + Q3)

La tabella sintetizza i 14 lavori estratti per Q1 (predictor sparsità, 7 paper) e Q3 (MoE locality, 7 paper). Ogni numero è etichettato **[MISURATO]** o **[PROIETTATO]** e associato alla rispettiva baseline. La colonna *Predictor sì/no + costo* esplicita il costo del predictor (parametri, latency, memory). La colonna *Granularità* riporta E/k per i MoE e la struttura di sparsità per i predictor. La mini-tabella dReLU (Q2) è nella sezione 2.4.

Tabella 1 — Predictor di sparsità (Q1) + Località esperti MoE (Q3): scala, architettura, numero misurato, predictor sì/no e costo, granularità.

Lavoro	Scala	Architettura	Numero misurato (baseline)	Predictor sì/no + costo	Granularità
DejaVu [1]	1.3B-175B (OPT, BLOOM, LLaMA)	Transformer decoder-only (ReLU nativo)	$1.8\text{--}2\times$ vs FasterTransformer, $4.8\text{--}6\times$ vs HF [MISURATO]; predictor accuracy 93-99% [MISURATO]	SÌ: 2-layer MLP predictor, 500 campioni C4; async lookahead; latency overhead non riportato ma <i>"may easily increase latency"</i>	Per-neuron (no BS8)
PowerInfer [2]	7B-175B (OPT, LLaMA, Falcon)	Transformer decoder-only (ReLU/ReLU/SiLU)	$11.69\times$ vs llama.cpp; 13.20 tok/s avg RTX 4090 OPT-175B [MISURATO]; predictor accuracy $\geq 95\%$ [MISURATO]	SÌ: adaptive MLP predictor ($\sim 10\%$ parametri totali), $<10\%$ latency overhead [MISURATO]	Per-neuron (no BS8)
PowerInfer-2 [3]	7B-47B (Llama-2, TurboSparse-Mixtral/Mixtral)	Transformer decoder-only dense + MoE	$29.2\times$ vs llama.cpp decoding; 11.68 tok/s TurboSparse-Mixtral-47B smartphone [MISURATO]	SÌ: riusa PowerInfer predictor; polymorphic neuron engine; segmented caching	Neuron cluster adattivo (no BS8 fisso)

Lavoro	Scala	Architettura	Numero misurato (baseline)	Predictor sì/no + costo	Granularità
Turbo Sparse [4]	300M (from-scratch) + 7B/47B (ReLUfication)	Transformer decoder-only dense + MoE (dReLU)	2-5× decoding speedup; CPU-only 2.08-2.81× vs llama.cpp [MISURATO]; dReLU → 90% FFN sparsity (vs 70% ReGLU)	SÌ: predictor per FFN block, per-MoE-expert su Mixtral (integrato PowerInfer); costo non riportato	Per-neuron (dReLU su gate E up)
ProSparse [5]	7B/13B (LLaMA2)	Transformer decoder-only (ReLU+FATReLU post-hoc)	≥3× vs llama.cpp (approximate); 4.65× LLaMA2-13B [MISURATO]; 89.32% sparsity (vs 70% vanilla ReLU); 2.44× single-step sparse GPU ops [MISURATO]	Approximate: SÌ (DejaVu-style); Accurate: NO predictor (sparse GPU operators); costo non riportato	Per-neuron (no BS8)
ReLU Strikes Back [6]	1.3B (from-scratch) + 6.7B/7B (ReLUfication)	Transformer decoder-only (ReLU/shifted ReLU) — OPT, Llama, Falcon	3× FLOPS reduction [MISURATO]; wall-clock non riportato (FLOPS proxy); aggregated sparsity ~50% (OPT-6.7B 150 token) [MISURATO]	NO predictor (esplicito: "_no need to train a separate prediction module"_); costo 0	Row-skipping (structured chunks, no BS8 esplicito)
SparseInfer [7]	7B/13B (ProSparse-Llama2)	Transformer decoder-only (ReLU via ProSparse)	1.79× (13B) / 1.74× (7B) vs llama.cpp; 1.27× / 1.30× vs PowerInfer; <1%p accuracy loss [MISURATO]	SÌ training-free: sign-bit XOR, 0 parametri aggiuntivi; 337.5MB vs 1480MB DejaVu (4.38× meno memoria); 4× meno latency overhead [MISURATO]	Sign-bit packing 32-elementi per int (no BS8)
Mixtral-offloading [8]	47B (Mixtral-8x7B)	Transformer MoE (E=8, k=2)	2× vs naive accelerate; 1.2-1.5× vs LRU-only [MISURATO]; 2.09-3.06 tok/s batch=1	SÌ: speculative expert loading (gating next-layer su hidden state current-layer); LRU cache k=2/4	E=8, k=2 (coarse; cache 2-4 esperti per layer)

Lavoro	Scala	Architettura	Numero misurato (baseline)	Predictor sì/no + costo	Granularità
Pre-gated MoE [9]	SwitchTransformer E=128 (riferimento)	MoE con pre-gate architetturale (richiede re-training)	23% overhead vs oracular GPU-only; 4.2× memory peak reduction [MISURATO]	Sì architetturale: pre-gate function nel blocco N per selezionare esperti del blocco N+1; non applicabile a modelli off-the-shelf	E=128, k=1 (SwitchTransformer ref)
MoE-Infinity [10]	Mixtral-8x7B, DeepSeek-V2-Lite (E=64,k=6), Switch (E=128,k=1), NLLB (E=128,k=2), Arctic	Transformer MoE (multi-granularità)	3.1-16.7× TPOT vs vLLM/DeepSpeed/Mixtral-Offloading/Ollama [MISURATO]; worst-case 1.4× su Mixtral E=8/k=2	Sì: trace-based prediction (K-means clustering Expert Activation Matrix); LRU esplicitamente scartato	Multi-granularità: E=8/64/128, k=1/2/6
Fiddler [11]	47B (Mixtral-8x7B)	Transformer MoE (E=8, k=2)	1.26× single-batch; 1.30× long-prefill; 11.57× beam search vs Mixtral-Offloading/DeepSpeed/llama.cpp [MISURATO]	NO prediction dinamica: popularity profiling offline + CPU-GPU orchestration statica; CPU kernel AVX512_BF16	E=8, k=2 (coarse; 256 esperti totali, 56-125 in GPU)
EdgeMoE [12]	0.4B-26B (Switch Transformer E=8/16/32/64/128, GPTSAN E=16)	Transformer MoE (Switch-style, k=1)	2.63-3.01× Jetson TX2 SSD; 4.49-5.43× Raspberry Pi 4B vs IO-EXP [MISURATO]	Sì: statistical profile inter-layer $P(E_n E_0..E_{n-1})$; 87.1% prediction confidence su ST-base-8 [MISURATO]	Switch E=8/16/32/64/128, k=1 (sparse; diverso da E=32/k=8)
SiDA-MoE [13]	0.22B-7.1B (Switch-base-8/64/128/256)	Transformer MoE (Switch-style, k=1)	2.60× / 3.93× throughput Switch-base-128/256 su SST2; 80% memory saving; <1% performance drop [MISURATO]	Sì: hash function + LSTM with sparse attention; hash hits rate = prediction accuracy	Switch E=8/64/128/256, k=1

Lavoro	Scala	Architettura	Numero misurato (baseline)	Predictor sì/no + costo	Granularità
Local Routing Consistency [14]	3B-57B (20 MoE LLM)	Multi-MoE (analisi empirica)	Mixtral-8x7B SRP@16=49.27 % $\approx$ i.i.d. 50% (-0.7 pp); PowerMoE E=40/k=8 SRP@16=54.69 % vs i.i.d. 40% (+14.7 pp); DeepSeekMoE E=64/k=6 SRP@16=36.79 % vs i.i.d. 18.75% (+18 pp) [MISURATO]	N/A (analisi, non sistema); misura località nativa, non predictor	20 modelli E=8-128, k=1-8; setup più vicino al nostro: PowerMoE E=40/k=8, OLMoE E=64/k=8

Fonti: [1]-[14], vedi Bibliografia. SRP@16 = Segment Routing Best Performance a $m=16$ token; i.i.d. base-rate = pk/E con $p=2$.

5. Domanda 4 — Prior-art da citare (lista secca raggruppata)

Lista secca di ~ 15 lavori che un reviewer considererebbe prior-art del nostro setup, raggruppati per 3 sotto-categorie come da brief: (a) sparsità dReLU predictor-free, (b) block-structured activation sparsity, (c) streaming di esperti in regime bandwidth-bound. Per ciascuno: 1 riga su cosa copre e cosa NON copre del nostro caso (5M, dReLU ternary, 92% hidden / 79% gate sparse, predictor FALSIFICATI a op-point pari, BS8 18% $\rightarrow$ 50% BPB-zero, persistenza temporale nulla, MoE E32/h128 top-8, working-set $\approx$ i.i.d., single-user CPU bandwidth-bound).

5.1 (a) Sparsità dReLU predictor-free

- **ReLU Strikes Back [6]** (Apple, ICLR 2024) — ReLU/shifted ReLU + inserted ReLU post-hoc, predictor-free esplicito. **COPRE**: predictor-free design (DIRECT MATCH), memory-bound framing, aggregated sparsity concept. **NON COPRE**: dReLU (usa ReLU), 5M scale (1.3B-7B), pesi ternari, BS8 esplicito, persistenza temporale = mask overlap (misura aggregated sparsity, metrica diversa), CPU consumer x86.
- **Turbo Sparse [4]** (SJTU/Tsinghua, giu 2024) — dReLU $\max(0, x \cdot w_{\text{gate}}) * \max(0, x \cdot w_{\text{up}})$ + continued pretraining 150B tokens, $\sim 90\%$ FFN sparsity. **COPRE**: dReLU activation (DIRECT MATCH), sparsità 90% (DIRECT MATCH con nostro 92%), MoE expert FFN sparsity, small-scale 300M validation. **NON COPRE**: 5M scale, pesi ternari, predictor-free (Turbo Sparse USA predictor), BS8 esplicito, persistenza temporale, CPU consumer x86.
- **ProSparse [5]** (Tsinghua, COLING 2025) — ReLU + L1 progressiva + FATReLU post-hoc, 89% sparsity, via *accurate* predictor-free (sparse GPU operators). **COPRE**: ReLUfication post-hoc, predictor-free via *accurate*, alta sparsità $\sim 89\%$. **NON COPRE**: dReLU (usa

ReLU+FATReLU), 5M scale (7B/13B), pesi ternari, BS8 esplicito, persistenza temporale, CPU consumer.

- **SparseInfer [7]** (Seoul Nat'l U, DAC 2025) — Training-free predictor basato su sign-bit XOR, 0 parametri aggiuntivi, 4.38× meno memoria di DejaVu. **COPRE**: training-free predictor (AFFINE al nostro *predictor falsificati*), sign-bit approach (bit-level, affine al nostro pshufb-LUT ternary), memory-bound mobile/edge. **NON COPRE**: 5M scale (7B/13B), dReLU (usa ReLU via ProSparse), pesi ternari, BS8, persistenza temporale, CPU consumer x86.
- **DejaVu [1]** (Stanford/UW/Together, ICML 2024) — Predictor 2-layer MLP, 1.8-2× wall-clock speedup su OPT-175B memory-bound. **COPRE**: memory-bound framing, predictor accuracy 93-99% su larga scala, sparsità intrinseca ReLU ~85-95%. **NON COPRE**: 5M scale (1.3B-175B), dReLU (usa ReLU nativo OPT), pesi ternari, BS8, persistenza temporale, CPU consumer. **CITARE COME PRIOR-ART DEL PREDICTOR-BASED APPROACH** (noi lo falsifichiamo).
- **PowerInfer [2]** (SJTU, SOSP 2024) — Adaptive MLP predictor (~10% parametri, <10% latency overhead), 11.69× vs llama.cpp. **COPRE**: memory-bound CPU-GPU offload, predictor overhead <10% misurato, consumer-grade hardware. **NON COPRE**: 5M scale (7B-175B), dReLU, pesi ternari, BS8, persistenza temporale, CPU-only single-thread. **CITARE COME PRIOR-ART DEL PREDICTOR-BASED APPROACH** + quantificazione costo predictor.

5.2 (b) Block-structured activation sparsity

- **ReLU Strikes Back [6]** (Apple, ICLR 2024) — Row-skipping structured chunks (NON BS8 fisso). **COPRE**: structured sparsity concept (row-level). **NON COPRE**: BS8 esplicito, BPB-zero cost, 5M scale.
- **PowerInfer-2 [3]** (SJTU, 2024) — Neuron cluster adattivo (dimensione determinata da XPU/memoria/I/O, NON BS8 fisso). **COPRE**: neuron cluster concept (prossimo al BS8). **NON COPRE**: BS8 fisso predictor-free, BPB-zero, 5M scale, dReLU ternary.
- **SparseInfer [7]** (Seoul Nat'l U, DAC 2025) — Sign-bit packing 32-elementi per int (NON BS8). **COPRE**: bit-level packing (affine al nostro pshufb-LUT ternary). **NON COPRE**: BS8 fisso, BPB-zero, 5M scale.
- **TernGEMM** (già citato in Dossier E5-bis [6]) — Ternary packed add/sub su pesi ternari, assembly ARM/Intel. **COPRE**: ternary CPU SIMD. **NON COPRE**: pshufb-LUT, BS8 esplicito activation sparsity, LLM, SSM, BPB.

Nota critica: NESSUN paper nella letteratura testa block-structured predictor-free BS8 esplicito con BPB-zero cost. Il nostro finding (BS8 skippable 18% → 50%, BPB zero) è **zona bianca** — territorio non coperto. I paper sopra citati sono i più vicini concettualmente ma operativamente diversi.

5.3 (c) Streaming di esperti in regime bandwidth-bound

- **Mixtral-offloading [8]** (MIPT/Yandex, dic 2023) — LRU cache + speculative expert loading (next-layer prediction), 2× vs naive accelerate. **COPRE**: batch=1 single-user, GPU consumer + DRAM offload, memory-bound, LRU + speculative prediction. **NON COPRE**: granularità

$E=8/k=2$ (coarse vs nostro $E=32/k=8$), CPU consumer x86, confronto esplicito con base-rate i.i.d., granular MoE.

- **MoE-Infinity [10]** (Edinburgh, ICML 2024) — Sparsity-Aware Expert Cache (trace-based prediction), $3.1\text{-}16.7\times$ TPOT vs vLLM/DeepSpeed. **COPRE**: batch=1 single-user, GPU consumer A5000, memory-bound, trace-based prediction, testato su DeepSeek-V2-Lite $E=64/k=6$ (granular affine al nostro). **NON COPRE**: hardware A5000 non x86 consumer, E32/h128 specifico, claim "25% per request" ambiguo.
- **Pre-gated MoE [9]** (KAIST/MSR, ISCA 2024) — Pre-gate architetturale per creare prefetchable locality, 23% overhead vs oracular GPU-only. **COPRE**: batch=1, memory-bound, offload CPU → GPU, prediction per prefetch. **NON COPRE**: richiede re-training (non applicabile off-the-shelf), hardware datacenter A100, non testa granular MoE E32/k8. **[ALLINEATO FORTE]** implicitamente conferma che località nativa è debole.
- **Fiddler [11]** (UW/Tsinghua, ICLR 2025) — Popularity profiling offline + CPU-GPU orchestration, $1.26\times$ single-batch. **COPRE**: GPU consumer/workstation + CPU + DRAM offload, batch=1, Mixtral $E=8/k=2$ (stesso base-rate 25%). **NON COPRE**: granularità $E=8$ (non $E=32$), popularity statica cross-request, non E32/h128, CPU kernel AVX512_BF16 (non x86 legacy).
- **EdgeMoE [12]** (BUP/Huawei, IEEE TMC 2025) — Statistical profile inter-layer $P(E_n | E_0..E_{n-1})$, $2.63\text{-}5.43\times$ vs IO-EXP. **COPRE**: on-device single-user, memory-bound, offload esterno, prediction inter-layer. **NON COPRE**: hardware edge ARM, Switch Transformer $k=1$ (diverso da $E=32/k=8$), statistical profile offline.
- **SiDA-MoE [13]** (Duke, MLSys 2024) — Hash + LSTM prediction, $2.60\text{-}3.93\times$ throughput. **COPRE**: batch=1, memory-bound, offload CPU → GPU, prediction-based prefetch. **NON COPRE**: hardware datacenter A100, Switch Transformer $k=1$, non granular MoE.
- **Local Routing Consistency [14]** (Fudan/USC/Huawei, 2025) — Analisi empirica località routing su 20 MoE LLM; Mixtral $\text{SRP}@16=49.27\% \approx \text{i.i.d. } 50\%$. **COPRE**: misura diretta località su setup granular affine (PowerMoE $E=40/k=8$, OLMoE $E=64/k=8$). **NON COPRE**: non è un sistema (non misura speedup), non E32/h128 specifico. **[ALLINEATO FORTE QUANTITATIVO]** col nostro finding working-set $\approx \text{i.i.d.}$
- **Switch Transformer [15]** (Google, JMLR 2022) — Fondativo: auxiliary load balancing loss forza uniform routing → working-set $\approx \text{i.i.d. BY DESIGN}$. **COPRE**: spiegazione architetturale del nostro finding. **NON COPRE**: non è un sistema di inference, non memory-bound profiling. **CITARE COME FONDAZIONE TEORICA** del perché working-set $\approx \text{i.i.d.}$ è conseguenza architetturale.
- **DeepSeekMoE [16]** (DeepSeek-AI, ACL 2024) — Architettura granular (mN esperti, mK attivi, K_s shared). **COPRE**: design granular affine al nostro E32. **NON COPRE**: non parla di caching/locality (paper architetturale). Local Routing Consistency mostra che questa architettura ha località PIÙ BASSA dei MoE coarse → rilevante per il nostro finding.

6. Contraddizioni e allineamenti con le nostre misure

Box espliciti per ciascuna contraddizione/allineamento tra le nostre misure e la letteratura. **Le nostre misure:** (a) pshufb-LUT 4.2-5× vs fp32 (bit-exact) sul MLP ternario su Zen 2 AVX2 no VNNI (dossier E5-bis); (b) sparsità intrinseca 92% hidden / 79% gate (misurata); (c) predictor FALSIFICATI a op-point pari (modello base $\geq$ base+regolarizzazione); (d) block-structured predictor-free BS8 skippable 18% $\rightarrow$ 50%, BPB zero; (e) persistenza temporale nulla (misurata due volte); (f) $+0.013 \pm 0.005$ BPB combinato ternary+dReLU a 5M; (g) MoE E32×h128 top-8 working-set $\approx$ i.i.d., persistenza = base-rate k/E, hot-pool falsificato, località solo da predizione.

6.1 Contraddizioni

[CONTRADDIZIONE POTENZIALE FORTE — ProSparse su L1 regolarizzazione]. ProSparse (COLING 2025) claim che L1 progressive regularization è NECESSARIA per spingere 70% (vanilla ReLU) $\rightarrow$ 89% senza perdita performance (ProSparse-7B 89.32% sparsity, 38.46 avg vs original-7B 37.96 — match). Customer finding: regolarizzazione NON serve (dReLU da solo dà 92%, base $\geq$ base+reg a op-point parity). **Numero nostro:** base $\geq$ base+reg, dReLU intrinseca 92%. **Numero paper:** L1 progressiva necessaria 70% $\rightarrow$ 89% su ReLU LLaMA2. **Possibile spiegazione (NON in paper):** ProSparse parte da SwiGLU LLaMA2 (40% sparsità intrinseca); customer parte da dReLU ternary (92% intrinseca) $\rightarrow$ regolarizzazione su base già sparsa può non dare ulteriore benefit. **Scale differenti** (7-13B vs 5M) e **architettura** (SwiGLU vs dReLU+ternary) rendono il confronto indiretto.

[CONTRADDIZIONE PARZIALE — MoE-Infinity su intra-request skewed-reuse]. MoE-Infinity (ICML 2024) claim skewed reuse intra-request (Fig. 2: Expert 2 reused >15 volte in Sequence 1). Customer finding: working-set $\approx$ i.i.d. $\rightarrow$ se la nostra misura è intra-request, c'è contraddizione. **Numero nostro:** working-set $\approx$ i.i.d. (persistenza = base-rate k/E). **Numero paper:** skewed reuse intra-request (Expert 2 reused 7× più di altri). **Caveat:** la loro misura è su workload specifico (Mixtral 8x7B su BigBench/MMLU/Flan); il nostro E32 granular potrebbe avere skewed reuse anche più debole (Local Routing Consistency mostra granular MoE = località più bassa). **Caveat 2:** lo stesso MoE-Infinity ammette "*after processing multiple requests, the skew disappears*" $\rightarrow$ **[ALLINEATO cross-request]**. La contraddizione è limitata al regime intra-request e probabilmente dipende dal workload specifico.

[CONTRADDIZIONE POTENZIALE — ReLU Strikes Back su aggregated sparsity]. ReLU Strikes Back (Apple, ICLR 2024) riporta aggregated sparsity NON-RANDOM (real reuse $>$ random reuse) per OPT-6.7B su WikiText: $\sim 50\%$ neuroni inutilizzati nei primi 150 token, aggregated $>$ random. Customer finding: persistenza temporale nulla (misurata due volte). **Numero nostro:** persistenza temporale = 0 (mask overlap token-to-token). **Numero paper:** aggregated sparsity $\sim 50\% >$ random (real reuse $>$ random). **MA:** aggregated sparsity è "*cumulative union di neuroni usati su t token*" — diverso da "*mask overlap tra token consecutivi*". Se customer misura overlap tra token consecutivi (e trova ~ 0), è compatibile con aggregated sparsity $>$ random: anche con ~ 0 overlap token-to-token, l'aggregated può crescere lentamente vs random. **Metrica diversa $\rightarrow$ non**

contraddizione diretta ma tensione concettuale. Reuse window $\gamma=16$ con slight PPL increase su OPT-6.7B — ma è un'approssimazione di inference, non misura di coerenza naturale.

[POTENZIALE CONTRADDIZIONE — Predictor-based speedup vs nostro predictor falsificati]. DeJaVu ($2\times$ wall-clock su OPT-175B), PowerInfer ($11.69\times$ su 7B-175B), Turbo Sparse ($2-5\times$ su 7B/47B), SparseInfer ($1.79\times$ su 7B/13B) tutti riportano speedup con predictor. Customer finding: predictor falsificati a op-point pari. **Numero nostro:** predictor non dà benefit a 5M CPU. **Numero paper:** $2-11.69\times$ speedup con predictor a 7B+ GPU. **Possibile spiegazione (NON in paper):** a 7B+ con GPU, il predictor ammortizza I/O cost di 7B+ pesi da trasferire CPU $\rightarrow$ GPU o da HBM; a 5M su CPU, l'I/O cost è marginale $\rightarrow$ predictor overhead dominante. **Ipotesi non verificata nei paper.** La divergenza è coerente col regime (5M CPU vs 7B+ GPU) ma non è una contraddizione formale — i paper non testano a 5M CPU.

6.2 Allineamenti

[ALLINEATO FORTE — ReLU Strikes Back su predictor-free]. ReLU Strikes Back (Apple, ICLR 2024) esplicitamente: *"there is no need to train a separate prediction module that complicates the computation graph, and using efficient ReLU layers can be enough"*. Customer finding: predictor falsificati a op-point pari. **Numero nostro:** predictor non serve, $\text{base} \geq \text{base} + \text{reg}$. **Numero paper:** predictor-free esplicito, $3\times$ FLOPS reduction via ReLU alone. **Convergenza concettuale forte** — unico paper esplicitamente predictor-free nella letteratura activation sparsity.

[ALLINEATO FORTE — Turbo Sparse su dReLU + 90% sparsity]. Turbo Sparse (giu 2024) usa $\text{dReLU} \max(0, x \cdot W_{\text{gate}}) * \max(0, x \cdot W_{\text{up}}) \rightarrow 90\%$ FFN sparsity su Mistral-7B, 97% su MoE Mixtral-47B. Customer finding: dReLU + 92% hidden / 79% gate sparse. **Numero nostro:** dReLU gated MLP ternary, 92% hidden / 79% gate. **Numero paper:** dReLU 90% FFN sparsity. **Convergenza diretta** — Turbo Sparse è l'unico paper che usa dReLU esplicitamente, e gli numeri di sparsità sono coerenti.

[ALLINEATO FORTE E QUANTITATIVO — Local Routing Consistency su working-set $\approx$ i.i.d.]. Local Routing Consistency (Liang et al., 2025) misura direttamente località di routing su 20 MoE LLM. Mixtral-8x7B ($E=8, k=2$, i.i.d. base-rate 50%): $\text{SRP@16}=49.27\% \rightarrow$ gap -0.7 pp (essenzialmente i.i.d.). Long-range ($m=256$, cache $2\times k$): $\text{SCH}=61.85\%$ (+12 pp sopra i.i.d. — località marginale). PowerMoE ($E=40, k=8$, setup più vicino al nostro): $\text{SRP@16}=54.69\%$ vs i.i.d. 40% (+14.7 pp). **Numero nostro:** working-set $\approx$ i.i.d., persistenza = base-rate k/E . **Numero paper:** Mixtral $\text{SRP@16} \approx$ i.i.d.; PowerMoE +14.7 pp sopra i.i.d. (marginale). **Allineamento quantitativo forte** — il nostro finding è coerente con la letteratura empirica più diretta.

[ALLINEATO FORTE — Switch Transformer su load balancing loss]. Switch Transformer (Fedus et al., JMLR 2022) adotta auxiliary load balancing loss (Eq. 4) che forza uniform routing $\rightarrow$ working-set $\approx$ i.i.d. BY DESIGN. **Numero nostro:** working-set $\approx$ i.i.d. **Numero paper:** load balancing loss standard practice forza uniform routing. **Spiegazione architetturale** del nostro finding — la maggior parte dei MoE moderni eredita questa pratica $\rightarrow$ il finding del cliente è conseguenza diretta del design, non anomalia.

[ALLINEATO — Convergenza su prediction-based prefetch]. TUTTI i 7 paper MoE che raggiungono speedup misurabili (Mixtral-offloading, Pre-gated MoE, MoE-Infinity, EdgeMoE, SiDA-MoE, Fiddler indirettamente) usano **predizione** (next-layer, cross-layer statistical, hash+LSTM, trace-based) — NESSUNO si affida a puro working-set locality / LRU. **Numero nostro:** località solo da predizione. **Numero paper:** 7/7 usano prediction. **VALIDA direttamente** il nostro finding — se LRU/working-set locality bastasse, non servirebbe prediction. Il fatto che Mixtral-offloading aggiunga speculative prediction oltre a LRU è coerente col nostro finding.

[ALLINEATO PARZIALE — SparseInfer su training-free predictor]. SparseInfer (DAC 2025) riconosce che training-based predictor è costoso (1480MB per Llama2-13B, retraining per ogni change) e propone training-free alternative (sign-bit XOR, 0 parametri). Customer va oltre: anche training-free predictor non serve, block-structured predictor-free è sufficiente. **Numero nostro:** predictor falsificati (anche training-free). **Numero paper:** training-based predictor costoso, training-free alternativo. **SparseInfer è un passo intermedio** verso customer's position.

[ALLINEATO PARZIALE — ProSparse su via accurate predictor-free]. ProSparse (COLING 2025) ha una via "accurate" con sparse GPU operators **SENZA predictor** — calcolo denso + skip degli zeri noti dall'output di ReLU. **Numero nostro:** block-structured predictor-free BS8, BPB zero. **Numero paper:** predictor-free via accurate (sparse GPU operators). **Concettualmente affine** sul principio predictor-free, ma non è BS8-structured e non misura BPB-zero.

[ALLINEATO — Granular MoE ha località più bassa]. Local Routing Consistency mostra DeepSeekMoE (granular $E=64/k=6 + 2$ shared) $SRP@16=36.79\% < \text{Mixtral-8x7B (coarse } E=8/k=2, \text{ no shared) } 49.27\%$. Customer setup E32 granular è più affine a DeepSeekMoE che a Mixtral → aspettiamo località ANCOR PIÙ VICINA a i.i.d. rispetto alla letteratura su Mixtral. **Numero nostro:** working-set $\approx$ i.i.d. su E32 granular. **Numero paper:** granular MoE ha località più bassa. **Coerente** — il nostro finding è atteso per design.

6.3 Sintesi — contraddizioni da dichiarare in review

Contraddizioni da dichiarare esplicitamente in review (in ordine di severità):

- **[CONTRADDIZIONE POTENZIALE FORTE] ProSparse su L1 reg:** paper claim L1 progressiva necessaria 70% → 89%; customer claim reg non serve (dReLU intrinseca 92%, base $\geq$ base+reg). Da dichiarare e motivare con architettura (SwiGLU+ReLU vs dReLU) e scala (7-13B vs 5M).
- **[CONTRADDIZIONE PARZIALE] MoE-Infinity su intra-request skewed-reuse:** paper claim skewed intra-request (Fig. 2); customer claim working-set $\approx$ i.i.d. Da dichiarare e motivare con workload (Mixtral 8x7B su BigBench/MMLU/Flan vs nostro E32 granular) + cross-request allineato ("*skew disappears after multiple requests*").
- **[CONTRADDIZIONE POTENZIALE] ReLU Strikes Back su aggregated sparsity:** paper claim aggregated > random (real reuse); customer claim persistenza temporale = 0. Da dichiarare e motivare con metrica diversa (aggregated cumulative union vs mask overlap token-to-token).

- **[POTENZIALE CONTRADDIZIONE] Predictor-based speedup vs predictor falsificati:** 4 paper (DejaVu, PowerInfer, Turbo Sparse, SparseInfer) riportano 2-11.69× speedup con predictor a 7B+ GPU; customer non vede benefit a 5M CPU. Da dichiarare e motivare con regime (5M CPU memory-bound vs 7B+ GPU memory-bound) — i paper non testano a 5M CPU, quindi non è contraddizione formale.

Allineamenti da citare a supporto (in ordine di forza):

- **[ALLINEATO FORTE] ReLU Strikes Back su predictor-free:** *"no need to train a separate prediction module"* — supporto diretto al nostro finding.
- **[ALLINEATO FORTE] Turbo Sparse su dReLU + 90% sparsity:** $\text{dReLU} \max(0, x \cdot W_{\text{gate}}) * \max(0, x \cdot W_{\text{up}}) \rightarrow 90\%$ FFN sparsity, coerente col nostro 92%.
- **[ALLINEATO FORTE E QUANTITATIVO] Local Routing Consistency su working-set $\approx$ i.i.d.:** Mixtral SRP@16=49.27% $\approx$ i.i.d. 50%; PowerMoE E=40/k=8 +14.7 pp sopra i.i.d. (marginale).
- **[ALLINEATO FORTE] Switch Transformer su load balancing loss:** spiega architetturalmente perché working-set $\approx$ i.i.d. (load balancing loss standard practice).
- **[ALLINEATO] Convergenza su prediction-based prefetch:** 7/7 paper MoE con speedup usano prediction, NESSUNO si affida a puro working-set locality / LRU $\rightarrow$ valida *"località solo da predizione"*.
- **[ALLINEATO PARZIALE] SparseInfer su training-free predictor + ProSparse via accurate:** passi intermedi verso predictor-free.
- **[ALLINEATO] Granular MoE ha località più bassa:** DeepSeekMoE SRP@16=36.79% < Mixtral 49.27% $\rightarrow$ il nostro E32 granular è atteso avere località ancora più vicina a i.i.d.

Conclusione fattuale. Nessuna contraddizione formale con i nostri numeri di speedup (4.2-5× vs fp32, dossier E5-bis), di costo BPB (+0.013±0.005 a 5M, +0.018-0.022 proiezioni ternarizzate), o di finding MoE (working-set $\approx$ i.i.d.). **Contraddizioni sostanziali da dichiarare:** ProSparse su L1 reg (architettura + scala diverse), MoE-Infinity su intra-request (workload + cross-request allineato), ReLU Strikes Back su aggregated sparsity (metrica diversa). **Allineamenti forti a supporto:** ReLU Strikes Back su predictor-free, Turbo Sparse su dReLU, Local Routing Consistency su working-set $\approx$ i.i.d. (quantitativo), Switch Transformer su load balancing loss (architetturale). **Zona bianca confermata:** nessun paper testa combinazione esatta del nostro setup (5M + dReLU ternary + 92% sparse + predictor-free BS8 + persistenza temporale nulla + E32/h128 granular MoE + single-user CPU bandwidth-bound).

7. Bibliografia

Bibliografia numerata. Formato: [N] Autori (Anno). Titolo. Venue/arXiv-id. URL. Le citazioni inline nel testo si riferiscono a questi numeri.

- [1] Liu, Z.; Wang, J.; Dao, T.; Zhou, T.; Yuan, B.; Song, Z.; Shrivastava, A.; Zhang, C.; Tian, Y.; Ré, C.; Chen, B. (Stanford; UW; Together AI) (2024). *Deja Vu: Contextual Sparsity for*

- Efficient LLMs at Inference Time*. ICML 2024; arXiv:2310.17157. URL: <https://arxiv.org/abs/2310.17157> — codice: <https://github.com/FMInference/DejaVu>.
- [2] Song, Y.; Mi, Z.; Xie, H.; Chen, H. (SJTU-IPADS) (2024). *PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU*. SOSP 2024; arXiv:2312.12456. URL: <https://arxiv.org/abs/2312.12456> — codice: <https://github.com/SJTU-IPADS/PowerInfer>.
- [3] Xue, Z.; Song, Y.; Mi, Z.; Chen, L.; Xia, Y.; Chen, H. (SJTU-IPADS) (2024). *PowerInfer-2: Fast Large Language Model Inference on a Smartphone*. arXiv:2406.06282. URL: <https://arxiv.org/abs/2406.06282> — sito: <https://powerinfer.ai/v2>.
- [4] Song, Y.; Xie, H.; Zhang, Z.; Wen, B.; Ma, L.; Mi, Z.; Chen, H. (SJTU-IPADS; Tsinghua) (2024). *Turbo Sparse: Achieving LLM SOTA Performance with Minimal Activated Parameters*. arXiv:2406.05955, giugno 2024. URL: <https://arxiv.org/abs/2406.05955> — modelli: <https://huggingface.co/PowerInfer>.
- [5] Song, C.; Han, X.; Zhang, Z.; Hu, S.; Shi, X.; Li, K.; Chen, C.; Liu, Z.; Li, G.; Yang, T.; Sun, M. (Tsinghua) (2025). *ProSparse: Introducing and Enhancing Intrinsic Activation Sparsity within Large Language Models*. COLING 2025; arXiv:2402.13516. URL: <https://arxiv.org/abs/2402.13516> — modelli: <https://huggingface.co/SparseLLM/prosparse-llama-2-7b>.
- [6] Mirzadeh, I.; Alizadeh, K.; Mehta, S.; Del Mundo, C.C.; Tuzel, O.; Samei, G.; Rastegari, M.; Farajtabar, M. (Apple) (2024). *ReLU Strikes Back: Exploiting Activation Sparsity in Large Language Models*. ICLR 2024; arXiv:2310.04564. URL: <https://arxiv.org/abs/2310.04564>.
- [7] Shin, J.; Yang, H.; Yi, Y. (Seoul National University) (2025). *SparseInfer: Training-free Prediction of Activation Sparsity for Fast LLM Inference*. DAC 2025; arXiv:2411.12692. URL: <https://arxiv.org/abs/2411.12692>.
- [8] Eliseev, A.; Mazur, D. (MIPT / Yandex SDA / Yandex Research) (2023). *Fast Inference of Mixture-of-Experts Language Models with Offloading*. arXiv:2312.17238, dicembre 2023. URL: <https://arxiv.org/abs/2312.17238> — repo: <https://github.com/dvmazur/mixtral-offloading>.
- [9] Hwang, R. (KAIST); Wei, J. (USTC/MSR); Cao, S.; Hwang, C.; Tang, X.; Cao, T.; Yang, M. (Microsoft Research) (2024). *Pre-gated MoE: An Algorithm-System Co-Design for Fast and Scalable Mixture-of-Expert Inference*. ISCA 2024; arXiv:2308.12066. URL: <https://arxiv.org/abs/2308.12066> — repo: https://github.com/ranggihwang/Pregated_MoE.
- [10] Xue, L.; Fu, Y.; Lu, Z.; Sun, C.; Mai, L.; Marina, M. (University of Edinburgh) (2024). *MoE-Infinity: Efficient MoE Inference on Personal Machines with Sparsity-Aware Expert Cache*. ICML 2024; arXiv:2401.14361. URL: <https://arxiv.org/abs/2401.14361> — repo: <https://github.com/EfficientMoE/MoE-Infinity>.
- [11] Kamahori, K.; Tang, T.; Gu, Y.; Zhu, K.; Kasikci, B. (UW; Tsinghua) (2025). *Fiddler: CPU-GPU Orchestration for Fast Inference of Mixture-of-Experts Models*. ICLR 2025; arXiv:2402.07033. URL: <https://arxiv.org/abs/2402.07033> — repo: <https://github.com/efeslab/fiddler>.
- [12] Yi, R.; Guo, L. (Huawei); Wei, S.; Zhou, A.; Wang, S.; Xu, M. (BUPT) (2025). *EdgeMoE: Fast On-Device Inference of MoE-based Large Language Models*. IEEE TMC 2025; arXiv:2308.14352. URL: <https://arxiv.org/abs/2308.14352>.

- [13] Du, Z.; Li, S.; Wu, Y.; Jiang, X.; Sun, J.; Zheng, Q.; Wu, Y.; Li, A.; Li, H.; Chen, Y. (Duke University et al.) (2024). *SiDA-MoE: Sparsity-Inspired Data-Aware Serving for Efficient and Scalable Large Mixture-of-Experts Models*. MLSys 2024; arXiv:2310.18859. URL: <https://arxiv.org/abs/2310.18859> — repo: <https://github.com/timlee0212/SiDA-MoE>.
- [14] Liang, J. (Fudan); Wang, S. (USC); Tian, M.; Li, Y.; Tang, D. (Huawei); Wei, Z. (Fudan) (2025). *Not All Models Suit Expert Offloading: On Local Routing Consistency of Mixture-of-Expert Models*. arXiv:2505.16056. URL: <https://arxiv.org/abs/2505.16056> — repo: <https://github.com/ljcleo/moe-lrc>.
- [15] Fedus, W.; Zoph, B.; Shazeer, N. (Google) (2022). *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*. JMLR vol. 23 (120), pp. 1-39, 2022; arXiv:2101.03961. URL: <https://arxiv.org/abs/2101.03961>.
- [16] Dai, D. et al. (DeepSeek-AI) (2024). *DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models*. ACL 2024; arXiv:2401.06066. URL: <https://arxiv.org/abs/2401.06066>.
- [17] Li, Y. et al. (2023). *Lina: Accelerating Relational Alignment in Multi-Agent Large Language Model Systems*. USENIX ATC 2023; arXiv:2210.17223. URL: <https://arxiv.org/abs/2210.17223>.